

操作系统试题

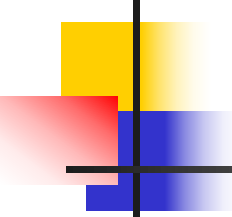
2013.11.23

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

1. SPOOLing系统是在主机控制下，通过通道把**I/O**工作脱机处理，**SPOOLing**不包括的程序是

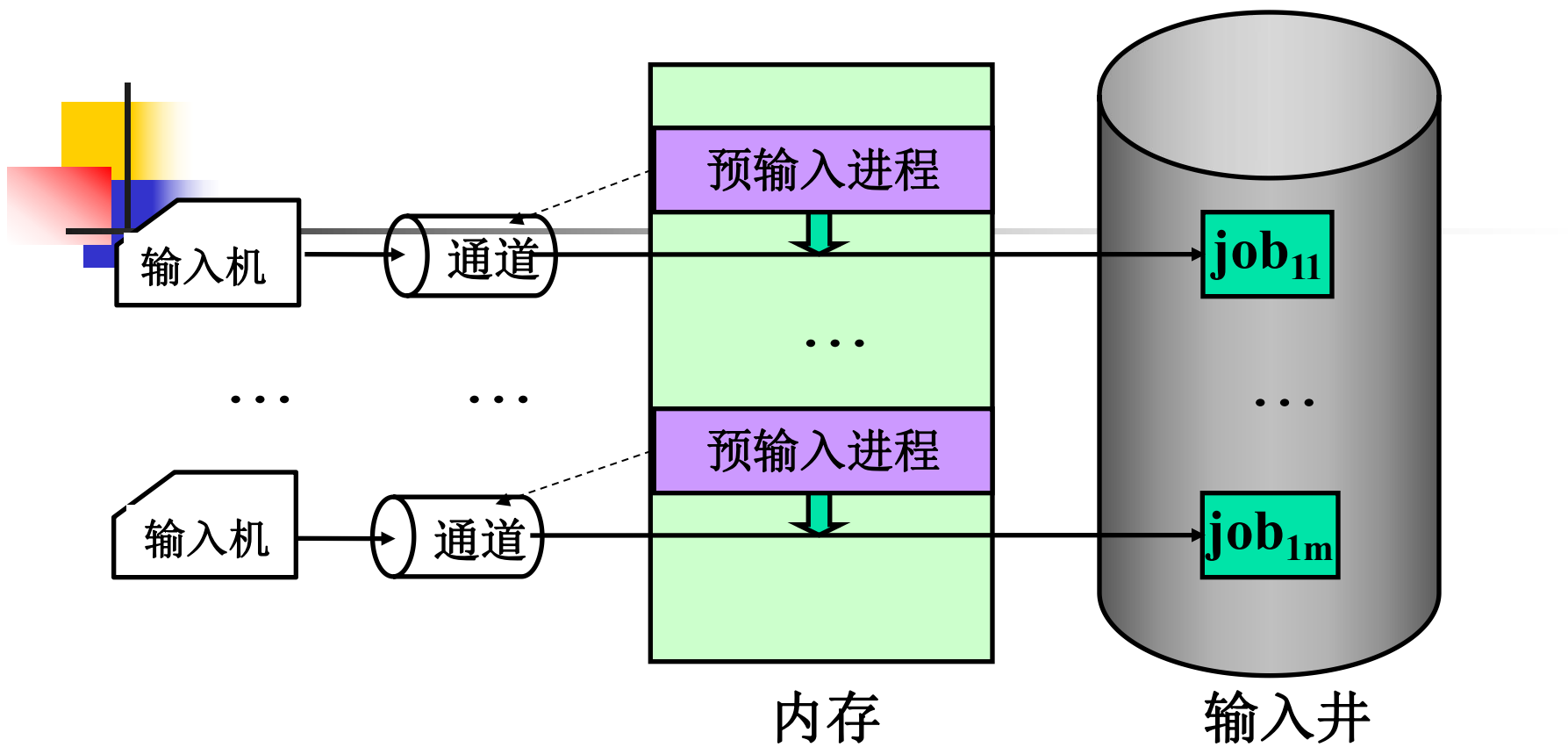
- A.** 预输入程序 **B.** 作业调度程序
- C.** 缓输出程序 **D.** 连接程序

知识点：**SPOOLing**系统

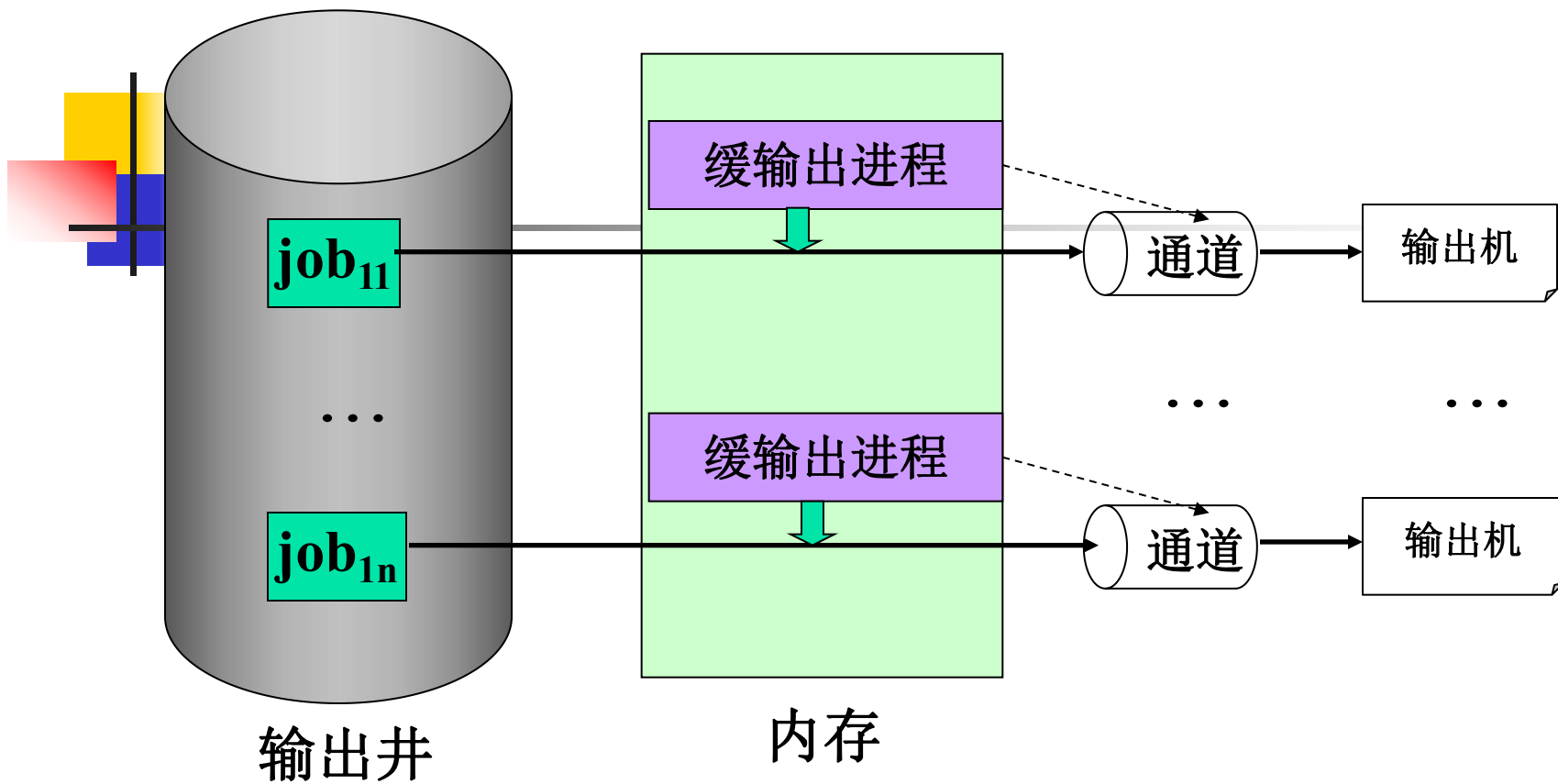


SPOOLing系统

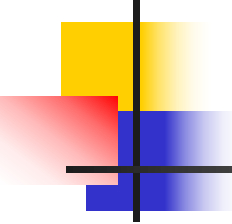
- **Spooling** 系统是实现虚拟设备的一个例子，是关于慢速字符设备如何与计算机主机交换信息的一种技术，通常称为“假脱机技术”。通过采用预输入和缓输出的方法，使用共享设备的一部分空间来模拟独占设备，以提高独占设备的利用率。
- **Spooling** 系统硬件部分包括输入机、输出机、通道、输入井和输出井。
 - 输入井和输出井：是在磁盘上开辟出来的两个存储区域。输入井模拟输入设备，用于存储I/O设备输入的数据。输出井模拟虚拟输出设备，用于存储用户程序的输出数据。
- **Spooling** 系统工作过程涉及到预输入进程、缓输出进程和作业调度程序。



SPOOLing输入程序 (1) vs. SPOOLing输入进程 (n)



SPOOLing输出程序 (1) vs. SPOOLing输出进程 (n)



SPOOLing系统

■ SPOOLing技术的特点:

- 提高了**I/O**速度: 将对低速**I/O**设备进行的**I/O**操作变为对输入井或输出井的操作,如同脱机操作一样,提高了**I/O**速度,缓和了**CPU**与低速**I/O**设备速度不匹配的矛盾.
- 设备并没有分配给任何进程: 在输入井或输出井中,分配给进程的是一存储区和建立一张**I/O**请求表.
- 实现了虚拟设备功能: 多个进程同时使用一独享设备,而对每一进程而言,都认为自己独占这一设备,不过,该设备是逻辑上的设备.

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

1. SPOOLing系统是在主机控制下，通过通道把**I/O**工作脱机处理，**SPOOLing**不包括的程序是

- A.** 预输入程序 **B.** 作业调度程序
- C.** 缓输出程序 **D.** 连接程序

答案 **D**

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

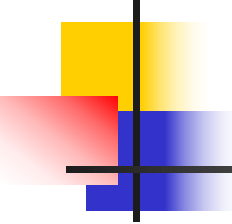
2. 计算机系统的下述机制中，

- I. 库函数 II. 终端命令
III. **GUI**界面 IV. 系统调用

属于操作系统提供给用户的接口是

- A.** I、II 和 III
B. I、II 和 IV
C. II、III 和 IV
D. I、III 和 IV

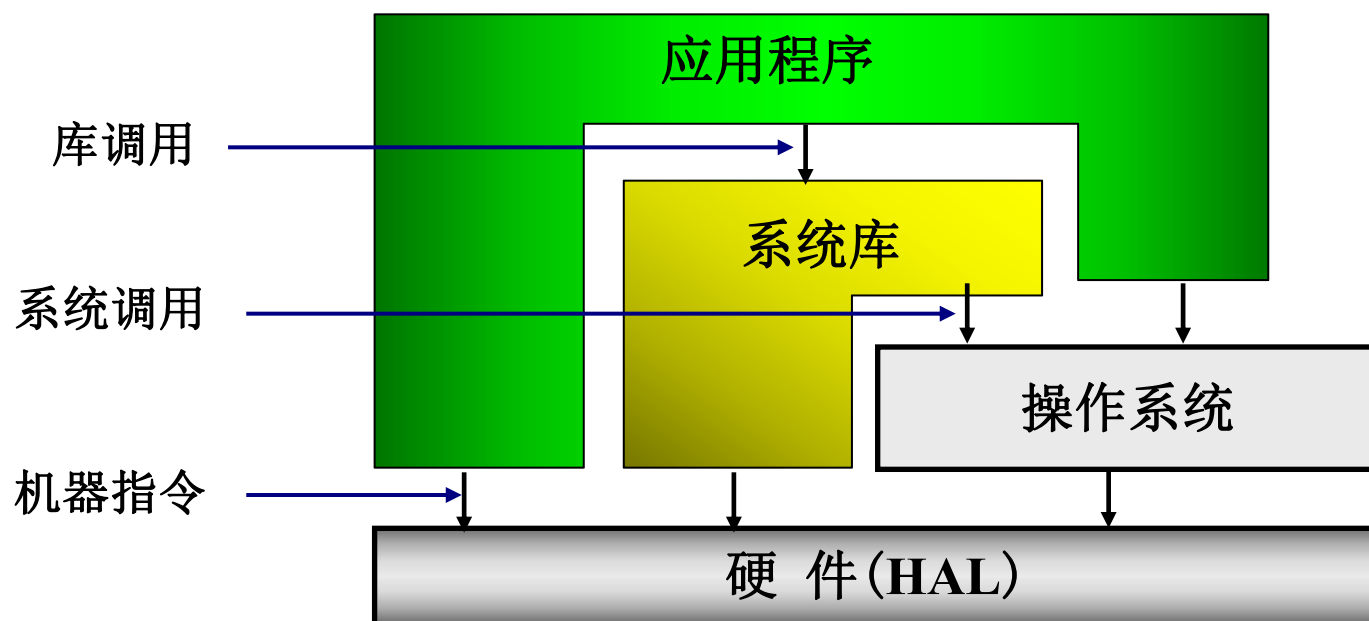
知识点：操作系统界面形式



操作系统界面形式

- 交互终端命令 (**Command Language**)
 - **Eg.UNIX shell**
 - **\$命令名 -选项 参数**
- 图形界面 (**GUI—Graphic User Interface**)
- 作业控制语言 (**Job Control Language**)
- 系统调用命令 (**OS API**)
 - 高级语言形式
 - **fd = open(file_name,mode)**
 - 汇编语言形式
 - 准备参数, **trap *n***, 取返回值

- 系统库(**lib**)可调用操作系统，执行硬件指令
- 应用程序可以调用**lib**和操作系统，执行硬件指令



一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

2. 计算机系统的下述机制中，

- I. 库函数 II. 终端命令
III. **GUI**界面 IV. 系统调用

属于操作系统提供给用户的接口是

- A.** I、II 和 III
B. I、II 和 IV
C. II、III 和 IV
D. I、III 和 IV

答案 C

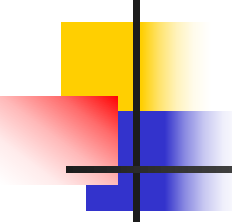
一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

3. 对如下中断事件

I. 时钟中断 II. 访管中断
III. 缺页中断 IV. 控制台中断
能引起外部中断的事件是

- A. I 和 II** **B. II 和 III**
C. III 和 IV **D. I 和 IV**

知识点：外部中断和内部中断



外部中断和内部中断

- 外部中断是可以屏蔽的中断，内部中断是不能屏蔽的。
 - 程序性中断、访管指令都属于内部中断。
 - 时钟中断和控制台中断是可以被屏蔽的，属于外部中断。

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

3. 对如下中断事件

I. 时钟中断 II. 访管中断
III. 缺页中断 IV. 控制台中断
能引起外部中断的事件是

- A. I 和 II** **B. II 和 III**
C. III 和 IV **D. I 和 IV**

答案 D

一、单项选择题（共30小题，每小题1分，共30分）

4. 设**int x;** 为定义的全局变量，两个进程**P1**和**P2**定义如下：
进程**P1**:

```
void main()
{ int m, n;
  x=1; m=0;
  if(x==1)
    m++;
  n=m;
  printf("n=%d\n", n);
}
```

进程**P2**:

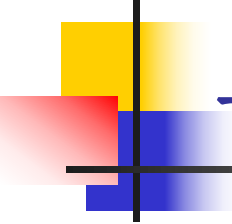
```
void main()
{ int m, n;
  x=0; m=0;
  if(x==0)
    m++;
  n=m;
  printf("n=%d\n", n);
}
```


一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

当运行语句 **cobegin P1; P2 coend;** 时，正确的说法是

- A. P1和P2的输出结果一定都是1;**
- B. P1输出结果一定为1， P2输出结果一定为0;**
- C. P1输出结果一定为0， P2输出结果一定为1;**
- D. P1和P2的输出结果不确定。**

知识点：与时间有关的错误 答案 D



与时间有关的错误

错误原因:

由于进程推进速度不一样, 导致进程执行交叉(**interleave**), 如果涉及公共变量(**x**), 那么可能发生与时间有关的错误。

Remarks:

某些交叉结果不正确;

必须去掉导致不正确结果的交叉。

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

5. 操作系统的文件管理中，文件控制块（**FCB**）的建立是

- A.** 在调用**creat()**时
- B.** 在调用**open()**时
- C.** 在调用**read()**时
- D.** 在调用**write()**时

知识点： FCB的创建与删除



文件控制块FCB (File Control Block)： 文件存在的标志，其中保存系统管理文件需要的全部信息

文件名
文件号
文件主
文件类型
文件属性
共享说明
文件长度
文件地址
建立日期
最后修改日期
最后访问日期
口令
其它

FCB创建： 建立文件时

FCB撤消： 删除文件时

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

5. 操作系统的文件管理中，文件控制块（**FCB**）的建立是

- A.** 在调用**creat()**时
- B.** 在调用**open()**时
- C.** 在调用 **read()**时
- D.** 在调用**write()**时

答案 A

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

6. 对系统的如下指标

I. 内存容量

II. 设备数量

III. **CPU**速度

IV. 中断响应时间

在多道程序设计中，道数限制要考虑的因素是

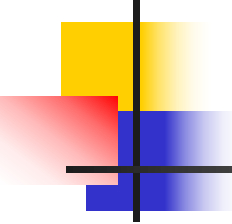
A. I 和 II

B. II 和 IV

C. III和IV

D. I 和IV

知识点：多道程序设计



多道程序设计

- 提高处理机、设备、内存等各种资源的利用率，从而提高系统效率。
- 增加同时运行程序的道数可以提高资源利用率，从而提高系统效率，但道数应与系统资源数量相当。
- 道数过少，系统资源利用率低。
- 道数过多，系统开销(**system overhead**)增大，程序响应速度下降。

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

6. 对系统的如下指标

I. 内存容量

II. 设备数量

III. **CPU**速度

IV. 中断响应时间

在多道程序设计中，道数限制要考虑的因素是

A. I 和 II

B. II 和 IV

C. III和IV

D. I 和 IV

答案 A

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

7. 下列选项中：

I. **I/O**请求 II. 时钟中断
III. **I/O**完成 IV. 设备进行**I/O**
可能引起进程切换的是

A. I、II和III **B.** II、III和IV
C. I、II和IV **D.** I、III和IV

知识点：进程切换



中断与处理机(进程)切换的关系

- 中断是处理机切换的必要条件，但不是充分条件
- 必然引起进程切换的中断
 - 进程自愿结束, `exit()`
 - 进程被强行终止;
 - 非法指令, 越界, `kill`
- 可能引起进程切换的中断
 - 时钟
 - 系统调用
 - 输入输出中断

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

7. 下列选项中：

I. **I/O**请求 II. 时钟中断
III. **I/O**完成 IV. 设备进行**I/O**
可能引起进程切换的是

A. I、II和III **B.** II、III和IV
C. I、II和IV **D.** I、III和IV

答案 A

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

8. 不属于强迫性中断的是

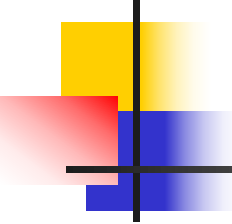
A. 内存校验错误

B. 越界中断

C. 缺页中断

D. 访管中断

知识点：中断类型



中断类型

■ 强迫性中断

- 运行程序不期望的
 - 时钟中断
 - **I/O**中断
 - 控制台中断
 - 硬件故障中断
 - **power failure**
 - 内存校验错
 - 程序性中断
 - 越界，越权
 - 缺页
 - 溢出，除**0**
 - 非法指令

■ 自愿性中断

- 运行程序期望的
 - 系统调用
 - 访管指令
 - 系统调用
 - **fd=open(fname,mode)**
 - 访管指令
 - 准备参数
 - **SVC n**
 - 取返回值

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

8. 不属于强迫性中断的是

A. 内存校验错误

B. 越界中断

C. 缺页中断

D. 访管中断

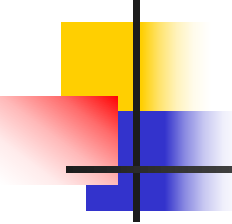
答案 D

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

9. 关于中断向量的错误论述是

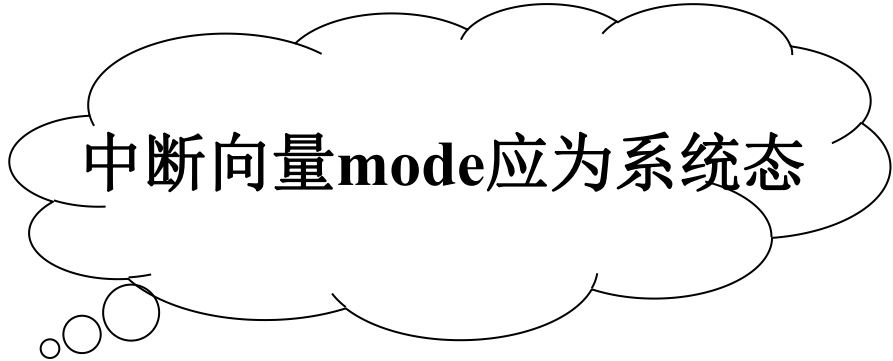
- A.** 中断向量保存中断处理程序的运行环境与入口地址(**PSW, PC**)。
- B.** 每个中断事件有一个中断向量。
- C.** 中断向量的存放位置是由硬件规定的。
- D.** 中断向量的内容是操作系统在系统初始化时设置好的。

知识点： 中断向量



中断向量

- 中断向量：中断处理程序的运行环境与入口地址（**PSW**，**PC**）
 - 每类中断事件有一个中断向量，
 - 中断向量的存放位置是由硬件规定的，
 - 中断向量的内容是**OS**在系统初始化时设置好的。



中断向量mode应为系统态

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

9. 关于中断向量的错误论述是

- A. 中断向量保存中断处理程序的运行环境与入口地址(**PSW, PC**)。
- B. 每个中断事件有一个中断向量。
- C. 中断向量的存放位置是由硬件规定的。
- D. 中断向量的内容是操作系统在系统初始化时设置好的。

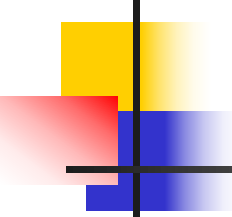
答案 B

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

10. 下列进程调度算法中，可能造成进程饿死的调度算法是

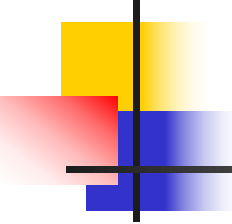
- A.** 循环轮换
- B.** 短进程优先
- C.** 先来先服务
- D.** 最高响应比优先

知识点：进程调度算法



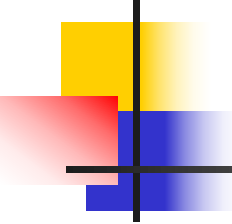
循环轮转算法

- 循环轮转算法：系统为每个进程规定一个时间片，所有进程按照其时间片的长短轮流运行，用完时间片后，如果还需要CPU时间到队列末尾排队。循环轮转算法是一种可剥夺调度策略，可以分为基本轮转和改进轮转：
 - 基本轮转：时间片(**quantum, time slice**)长度固定，不变；所有进程等速向前推进
 - 改进轮转：时间片长度不定，可变
- 特点
 - 如时间片过长，则会影响系统的响应速度
 - 如时间片过短，则会频繁地发生进程切换，增加系统开销
 - 适用于分时系统，具有公平、响应及时等特点



短作业(进程)优先

- 按照**CPU**的阵发时间递增的次序调度。
- 特点：
 - 假定所有任务同时到达，平均等待时间最短。
 - 长作业可能被饿死，即一个较长的就绪任务(作业)可能由于短作业的不断到达而长期的得不到运行机会，发生饥饿，甚至被饿死。



先到先服务算法

- **FCFS (First Come First Serve)**

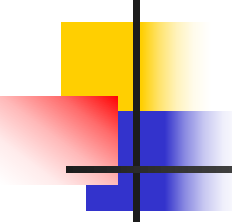
- 按进程申请**CPU**（就绪）的次序，即进入就绪态的次序调度。

- 优点：

- “公平”，不会出现饿死情况；

- 缺点：

- 短作业等待时间长，从而平均等待时间较长。



最高响应比优先(HRN)

- **HRN**是先到先服务算法和最短作业优先算法的折中，响应比计算公式：
 - **$RR = (BT + WT) / BT = 1 + WT / BT$**
 - 其中：
 - **BT=burst time**
 - **WT=wait time**
 - 优点：
 - 同时到达任务, 短者优先
 - 长作业随等待时间增加响应比增加，因而不会出现饿死现象

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

10. 下列进程调度算法中，可能造成进程饿死的调度算法是

- A.** 循环轮换
- B.** 短进程优先
- C.** 先来先服务
- D.** 最高响应比优先

答案 B

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

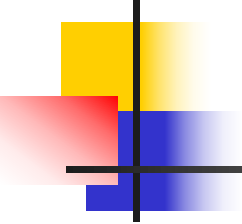
11. 关于进程切换有如下论述

- I. 根据系统栈保存下降进程的现场。
- II. 根据**PCB**保存下降进程的现场。
- III. 根据系统栈恢复上升进程的现场。
- IV. 根据**PCB**恢复上升进程的现场。

其中论述正确的是

- A.** I 和III
- B.** I 和IV
- C.** II 和III
- D.** II 和 IV

知识点：进程切换

- 
-
- 进程切换伴随着系统栈的切换，发生进程切换时，下降进程的现场信息从系统栈中弹出，保存到下降进程的**PCB**中。上升进程的现场信息从上升进程的**PCB**中恢复。

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

11. 关于进程切换有如下论述

- I. 根据系统栈保存下降进程的现场。
- II. 根据**PCB**保存下降进程的现场。
- III. 根据系统栈恢复上升进程的现场。
- IV. 根据**PCB**恢复上升进程的现场。

其中论述正确的是

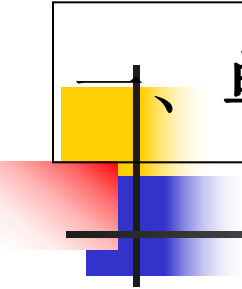
- A.** I 和III
- B.** I 和IV
- C.** II 和III
- D.** II 和 IV

答案 B

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

- 12.** 下列选项中，降低进程优先级的合理时机是
- A.** 进程的时间片用完
 - B.** 进程等待**I/O**完成进入就绪队列
 - C.** 进程在就绪队列中超过时限
 - D.** 进程从就绪转为运行

知识点：进程优先级



一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

分析：

- A.** 进程的时间片用完：进程刚刚占用完**CPU**，可以降低其优先级，以给其它进程运行机会
- B.** 进程等待**I/O**完成进入就绪队列：进程已经等待了一段时间，合理的做法应该是提高优先级或优先级不变，而不是降低优先级
- C.** 进程在就绪队列中超过时限：为了解决饥饿现象，实现公平，进程在就绪队列中超时应该提高优先级。
- D.** 进程从就绪转为运行：进程已经占有处理机运行了，没有必要改其优先级。

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

- 12.** 下列选项中，降低进程优先级的合理时机是
- A.** 进程的时间片用完
 - B.** 进程等待**I/O**完成进入就绪队列
 - C.** 进程在就绪队列中超过时限
 - D.** 进程从就绪转为运行

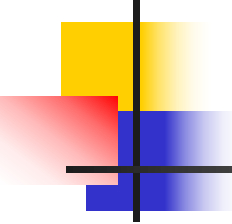
答案 A

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

13. 在多级中断系统中，多层嵌套中断的最内层中断处理结束后，以下论述中正确的论述是

- A.** 如果该中断是强迫性中断，则需要进程切换。
- B.** 如果该中断是自愿性中断，则需要进程切换。
- C.** 无论该中断是强迫性中断还是自愿性中断，都需要进程切换。
- D.** 无论该中断是强迫性中断还是自愿性中断，都不需要进程切换。

知识点：中断嵌套



中断嵌套

- 中断嵌套是指在中断处理过程中，响应新的中断称为中断嵌套。
- 一般原则：
 - 高优先级别中断可以嵌入低优先级中断
- 实现方法：
 - 中断响应后立即屏蔽不高于当前中断优先级的中断源。
- 当发生中断嵌套时，系统栈中保存的是中断处理程序的现场信息，所以最内层中断处理完毕后，恢复的是上一层中断的现场信息，而不需要进程切换。

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

13. 在多级中断系统中，多层嵌套中断的最内层中断处理结束后，以下论述中正确的论述是

- A.** 如果该中断是强迫性中断，则需要进程切换。
- B.** 如果该中断是自愿性中断，则需要进程切换。
- C.** 无论该中断是强迫性中断还是自愿性中断，都需要进程切换。
- D.** 无论该中断是强迫性中断还是自愿性中断，都不需要进程切换。

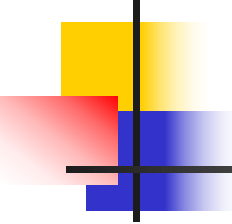
答案 D

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

14. 设与某类资源**R**相关联的信号量**S** 的初值为**3**，**S**当前值为**-2**。若**M**表示**R**的可用个数，**N**表示等待**R**的进程数，则当前**M**、**N**分别是

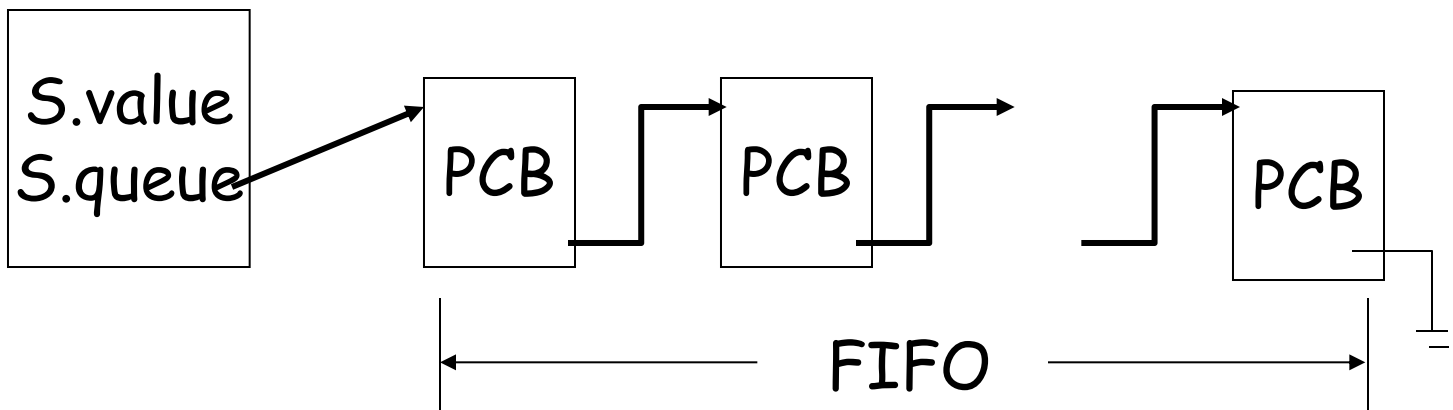
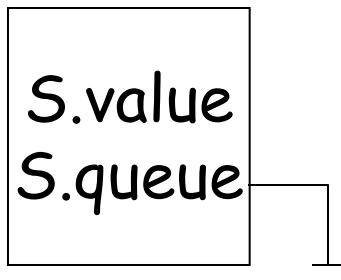
A. 3、0 B. 0、3 C. 0、2 D. 2、0

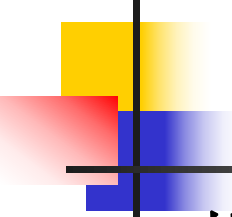
知识点：信号灯与**PV**操作



信号灯变量

Var S:semaphore;





P操作原语

P操作原语:

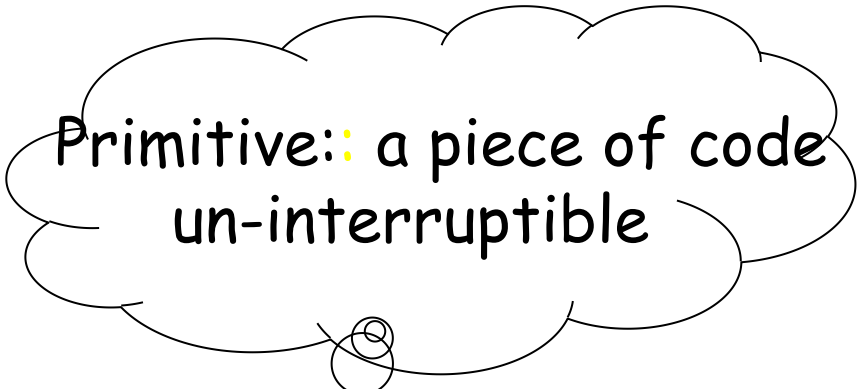
Procedure P(var s:semaphore)

s.value:=s.value-1;

If s.value<0 Then

asleep(s.queue)

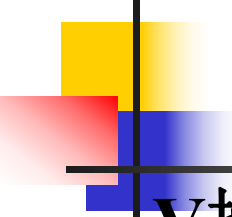
End



Primitive: a piece of code
un-interruptible

asleep(s.queue):

- (1) 执行此操作进程的PCB入s.queue尾（状态改为等待）；
- (2) 转处理机调度程序。



V操作原语

V操作原语:

Procedure V(var s:semaphore)

s.value:=s.value+1;

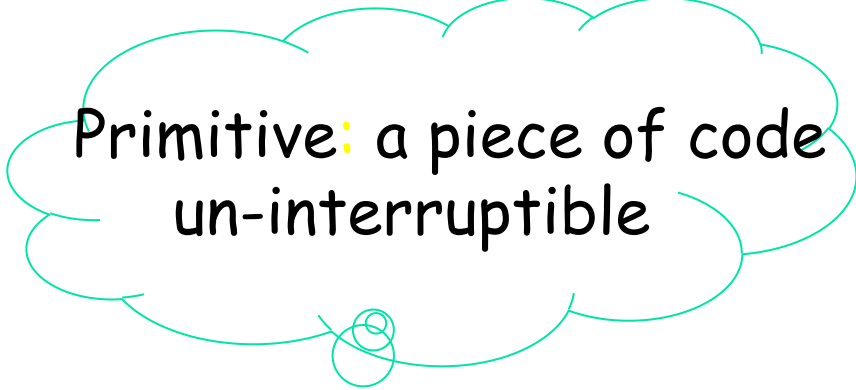
If s.value<=0 Then

wakeup(s.queue)

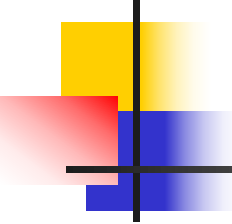
End

wakeup(s.queue)

s.queue链头PCB出等待队列，进入就绪队列（状态改为就绪）。



Primitive: a piece of code
un-interruptible



规定和结论

- 对于信号灯变量的规定：
 - 必须置一次初值，只能置一次初值，初值 ≥ 0 ;
 - 只能执行**P**操作和**V**操作，所有其它操作非法。
- 几个有用的结论：
 - 当 $s.value \geq 0$ 时， $s.queue$ 为空;
 - 当 $s.value < 0$ 时， $|s.value|$ 为队列 $s.queue$ 的长度;
 - 当 $s.value_{初}=1$ 时，可以实现进程互斥;
 - 当 $s.value_{初}=0$ 时，可以实现进程同步。

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

14. 设与某类资源**R**相关联的信号量**S** 的初值为**3**，**S**当前值为**-2**。若**M**表示**R**的可用个数，**N**表示等待**R**的进程数，则当前**M**、**N**分别是

A. 3、0 B. 0、3 C. 0、2 D. 2、0

答案 C

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

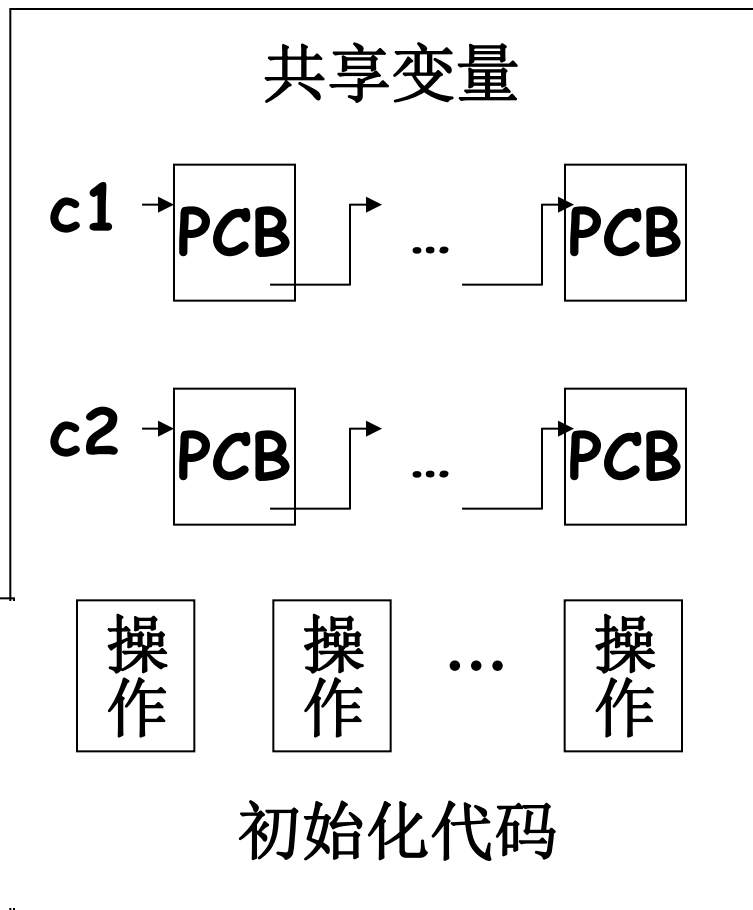
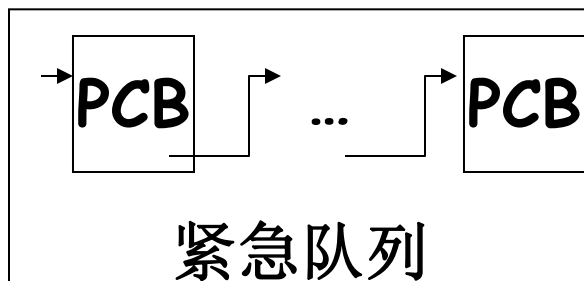
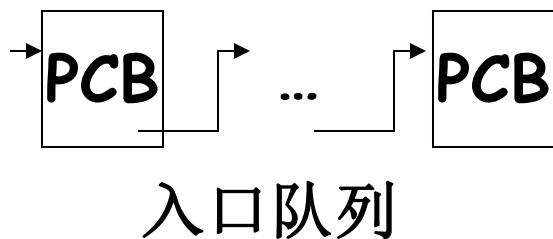
15. 在**Hoare**管程中，设某管程当前入口等待队列**EQ**中有进程**P0**、紧急等待队列**UQ**中有进程**P1**、条件变量**C**的等待队列**CQ**中有进程**P2**，进程**P3**拥有该管程的互斥权。当依次：进程**P4**要申请该管程互斥权、**P3**执行**signal (C)** 后，该管程各队列中的进程和运行进程是

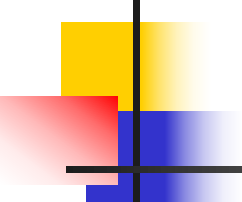
一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

- A. EQ中有P0，UQ中有P4，CQ中有P2、P3；P1运行**
- B. EQ中有P0、P4，UQ中有P1，CQ中有P3；P2运行**
- C. EQ中有P0、P4，UQ中有P2、P3，CQ为空；P1运行**
- D. EQ中有P0、P4，UQ中有P1、P3，CQ为空；P2运行**

知识点：Hoare管程

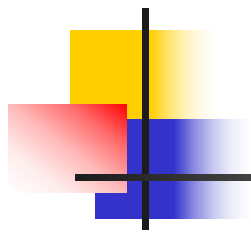
管程成分





■ 管程:

- **Wait(c):** 进程在管程中执行，当某个条件不满足时，执行**wait**操作，执行此操作的进程进入到对应的条件等待队列。同时判断紧急等待队列是否有进程，如果有，唤醒紧急等待队列中的一个进程，否则唤醒入口等待队列中的一个进程，并释放管程使用权。
- **Signal(c):** 进程在管程中执行，当某个条件发生时，就会执行**signal**操作，唤醒对应条件等待队列中的一个进程。此时管程中会有**2**个活动进程，这是不允许的，因为管程是临界区，要求互斥的进入管程。后续处理常用的有**2**种方式，分为**Hoare**管程和**Hansen**管程。



Hoare管程的处理方式是指从条件队列中被唤醒的进程继续执行，执行唤醒操作的进程进入到紧急等待队列。当它从紧急队列被唤醒后，继续执行管程内的其它代码。

Hansen管程的处理方式是被唤醒的进程继续执行，执行唤醒操作的进程离开管程，因为**Signal**是管程中的最后一条指令。

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

15. 在**Hoare**管程中，设某管程当前入口等待队列**EQ**中有进程**P0**、紧急等待队列**UQ**中有进程**P1**、条件变量**C**的等待队列**CQ**中有进程**P2**，进程**P3**拥有该管程的互斥权。当依次：进程**P4**要申请该管程互斥权、**P3**执行**signal (C)** 后，该管程各队列中的进程和运行进程是

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

- A. EQ中有P0，UQ中有P4，CQ中有P2、P3；P1运行**
- B. EQ中有P0、P4，UQ中有P1，CQ中有P3；P2运行**
- C. EQ中有P0、P4，UQ中有P2、P3，CQ为空；P1运行**
- D. EQ中有P0、P4，UQ中有P1、P3，CQ为空；P2运行**

答案 D

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

16. 某计算机系统中有**6**台打印机，多个进程均最多需要**2**台打印机，规定每个进程一次仅允许申请一台打印机。为保证一定不发生死锁，则允许参与打印机资源竞争的最大进程数是

A. 3 B. 4 C. 5 D. 6

知识点：同种组合资源死锁的必要条件



同种组合资源死锁的必要条件

M: 资源数量

N: 使用该类资源进程的数量

Σ : 所有进程所需要该类资源的总量

假定死锁, n 个进程参与了死锁($2 \leq n \leq N$)

参与死锁的进程所需资源的总量 $\geq M + n$

未参与死锁进程所需资源的总量 $\geq N - n$

所有进程所需资源的总量 $\Sigma \geq M + n + N - n = M + N$

当 $\Sigma < M + N$ 时, 一定没有死锁;

当 $\Sigma \geq M + N$ 时, 至少有一个交叉有死锁。

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

16. 某计算机系统中有**6**台打印机，多个进程均最多需要**2**台打印机，规定每个进程一次仅允许申请一台打印机。为保证一定不发生死锁，则允许参与打印机资源竞争的最大进程数是

A. 3 B. 4 C. 5 D. 6

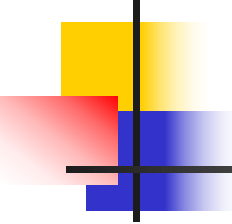
答案 C

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

17. 操作系统为实现多道程序并发，对内存管理可以采用多种方式，其中代价最小的是

- | | |
|----------------|-----------------|
| A. 分区管理 | B. 分页管理 |
| C. 分段管理 | D. 段页式管理 |

知识点：存储管理方式



存储管理方式

- 界地址管理方式（一维地址）：分区管理
- 页式管理方式（一维地址）：分页管理
- 段式管理方式（二维地址）：分段管理
- 段页式管理方式（二维地址）：段页式管理

页式管理、段式管理和段页式管理需要额外的内存空间保存段表和页表。界地址管理方式没有段表和页表，所以相比较代价较小。此外，页式管理、段式管理和段页式管理的地址变换过程比分区管理复杂。

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

17. 操作系统为实现多道程序并发，对内存管理可以采用多种方式，其中代价最小的是

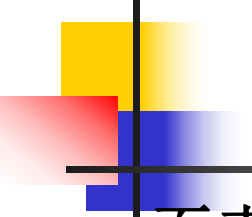
- | | |
|----------------|-----------------|
| A. 分区管理 | B. 分页管理 |
| C. 分段管理 | D. 段页式管理 |

答案 A

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

- 18.** 在页式存储管理中，每个页表的表项实际上是用于实现
- | | |
|------------------|-----------------|
| A. 访问内存单元 | B. 静态重定位 |
| C. 动态重定位 | D. 装载程序 |

知识点：页表



页表，每个进程一个，用于记录进程的逻辑页面与内存页框之间的对应关系。根据页号可以找到页框号。

逻辑页号:

0

1

2

3

页框号

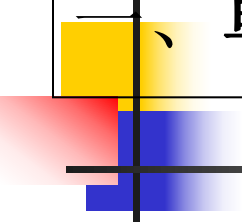
15

22

16

32

页框号是物理地址的高位部分,根据页框号与页内地址可以确定内存物理地址



一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

- 重定位：被换出的进程再次运行之前必须重新装入内存，而再次进入内存时的存放位置与换出之前通常不同，这就要求程序编址与内存存放位置无关，这种程序称为可重定位程序。
- 动态重定位：在进程运行时进行
- 静态重定位：在进程运行前编译时或装入时进行

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

18. 在页式存储管理中，每个页表的表项实际上是用于实现

- | | |
|------------------|-----------------|
| A. 访问内存单元 | B. 静态重定位 |
| C. 动态重定位 | D. 装载程序 |

答案 A

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

19. 某系统用位示图管理内存，位示图定义为 **char bitmap[400]**。页框号为**380**对应 **bitmap**的位置是

- A. bitmap[46] 的第3位**
- B. bitmap[46] 的第4位**
- C. bitmap[47] 的第3位**
- D. bitmap[47] 的第4位**

知识点：位示图

位示图 (bit map)

用一个bit代表一页状态，0表空闲，1表占用。（多单元）

1	0	0	...	1	...	1	0
---	---	---	-----	---	-----	---	---

第0页
第1页
第2页

...

第n页

...

第m页

分配：自头寻找第一个为0的位，改为1，返回页号；

去配：页号对应的位(bit)置为0。

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

19. 某系统用位示图管理内存，位示图定义为 **char bitmap[400]**。页框号为**380**对应 **bitmap**的位置是

- A. bitmap[46] 的第3位**
- B. bitmap[46] 的第4位**
- C. bitmap[47] 的第3位**
- D. bitmap[47] 的第4位**

答案 C

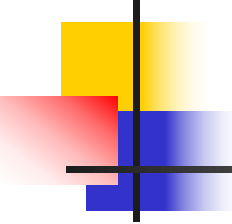
一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

20. 设磁盘格式化时把每磁道等分为**8**个扇区，磁盘转速为**5000**转/分钟。则（忽略启动时间）读取一个扇区所花费时间是

- A. 0.05 ms B. 0.15 ms**
C. 0.25ms D. 0.35ms

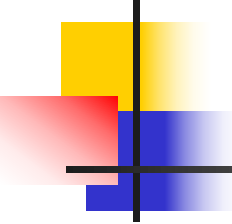
知识点：磁盘**I/O**参数

答案 B



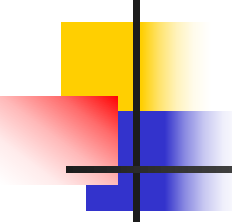
磁盘I/O参数

- 首先分析一下读/写一个磁盘块需要多少时间。它一般由如下三个因素确定：
 - 寻道时间（**seek time**）：将磁盘引臂移动到指定柱面所需要的时间；
 - 旋转延迟（**rotational delay**）：指定扇区旋转至磁头下的时间；
 - 传输时间（**transfer time**）：读/写一个扇区的时间。



磁盘I/O参数

- 寻道时间 T_s 计算公式如下：
 - $T_s = m \times n + s$
 - 其中， n 为跨越磁道数， m 为跨越一个磁道所用时间， s 为启动时间。
- 旋转延迟 T_r 计算公式如下：
 - $T_r = 1/(2r)$
 - 其中， r 为磁盘转速。该公式给出的是平均旋转延迟，它是磁盘旋转一周时间的一半，即旋转半周所花费的时间。



磁盘I/O参数

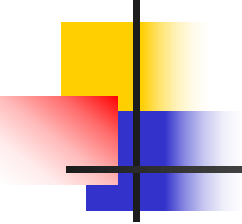
- 传输时间 Tt 计算公式如下：
 - $Tt = b / (rN)$
 - 其中， b 为读/写字节数， r 为磁盘转速， N 为一条磁道上的字节数。

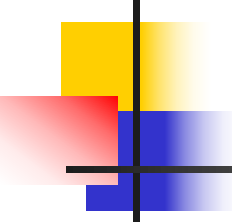
一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

21. 在动态异长分区的存储分配算法中，能保证空闲区按地址均匀分布的分配算法是

- A. First Fit**算法 **B. Next Fit**算法
C. Best Fit算法 **D. Worst Fit**算法

知识点：动态异长分区的分配

- 
-
- 动态异长分区的分配
 - 最先适应 (**First Fit**)
 - 最佳适应 (**Best Fit**)
 - 最坏适应 (**Worst Fit**)
 - 下次适应 (**Next Fit**)



最先适应算法 (First Fit)

空闲区首址	空闲区长度
128	64
256	32
1024	256
	0
...	...

空闲区：首址递增排列；

申请：取第一个可满足区域；

优点：尽量使用低地址空间，
高地址区保持大空闲区域

缺点：可能分割大空闲区。

Eg. 申请32将分割第
一个区域。

最佳适应算法 (Best Fit)

空闲区首址	空闲区长度
128	64
256	32
1024	256
	0
...	...

空闲区：首址递增排列；

申请：取最小可满足区域；

优点：尽量使用小空闲区，
保持大空闲区。

缺点：可能形成碎片
(fragment)。

Eg. 申请30将留下长
度为2的空闲区。



最坏适应算法 (Worst Fit)

空闲区首址	空闲区长度
128	64
256	32
1024	256
	0
...	...

空闲区：首址递增排列；
申请：取最大可满足区域；
优点：防止形成碎片。
缺点：分割大空闲区域。

下次适应算法 (Next Fit)

空闲区首址	空闲区长度
128	64
256	32
1024	256
	0
...	...

空闲区：首址递增排列；

申请：自上次分配空闲区域的下一个位置开始，选取第一个可满足的空闲区域；

优点：减少查找空闲区域所花费的时间开销，并使得空闲区域分布更均匀。

缺点：分割大空闲区域。

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

- 21.** 在动态异长分区的存储分配算法中，能保证空闲区按地址均匀分布的分配算法是
- A. First Fit算法 B. Next Fit算法**
C. Best Fit算法 D. Worst Fit算法

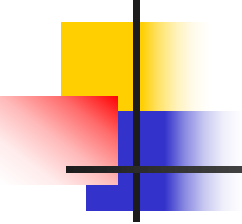
答案 B

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

22. 采用段式存储管理的系统中，若地址用**24**位表示，其中**8**位表示段号，则允许程序每个逻辑段的最大相对地址是：

- A. 2^{24}** **B. $2^{24}-1$**
C. 2^{16} **D. $2^{16}-1$**

知识点：段式存储管理的逻辑地址



段式存储管理的逻辑地址

逻辑地址=

段号	段内地址
----	------

 (二维地址)

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

22. 采用段式存储管理的系统中，若地址用**24**位表示，其中**8**位表示段号，则允许程序每个逻辑段的最大相对地址是：

A. 2^{24}

B. $2^{24}-1$

C. 2^{16}

D. $2^{16}-1$

答案 D

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

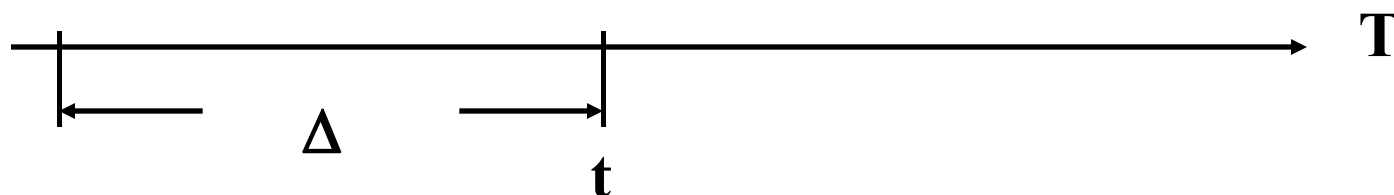
- 23.** 假设虚拟页式存储管理采用工作集模型。如果在 Δ 周期内确定某进程的工作集大小为**n**，则**n**的含义是
- A.** 该进程在 Δ 周期内淘汰页面的个数
 - B.** 该进程在 Δ 周期内访问页面的个数
 - C.** 该进程在 Δ 周期内发生缺页的次数
 - D.** 该进程在 Δ 周期内访问页面的次数

知识点：工作集模型

工作集模型(working set model)

工作集(working set): 进程在一段时间内所访问页面的集合。

...2 6 1 5 7 7 7 7 5 1 6 2 2 1 2 3 ... (page reference)



$$WS(t, \Delta) = \{5, 7, 1, 6, 2\}$$

Δ : 称为窗口尺寸(window size)。

Denning 认为: 为使程序有效运行, 工作集应能放入内存。

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

- 23.** 假设虚拟页式存储管理采用工作集模型。如果在 Δ 周期内确定某进程的工作集大小为**n**，则**n**的含义是
- A.** 该进程在 Δ 周期内淘汰页面的个数
 - B.** 该进程在 Δ 周期内访问页面的个数
 - C.** 该进程在 Δ 周期内发生缺页的次数
 - D.** 该进程在 Δ 周期内访问页面的次数

答案 B

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

- 24.** 文件系统中，把**FCB**分为次部和主部的好处是
- A.** 提高文件的查找速度
 - B.** 减少**FCB**所占空间
 - C.** 防止进程修改**FCB**信息
 - D.** 减少文件**I/O**操作的时间

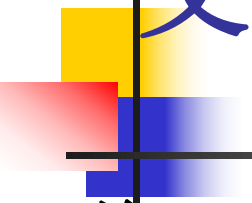
知识点：**FCB**的实现



FCB文件控制块的实现

FCB

- 次部：仅包括文件名称和标识文件主部的文件号。保存在目录文件中（目录文件在文件区）。
- 主部：包括除文件名称之外的所有信息和一个标识该主部与多少次部相对应的连接计数。当连接计数为**0**时，表示是一个空闲未用的**FCB**主部。**FCB**主部保存在外存**inode**区域，打开时读入内存。
- 在外存**inode**区域, **FCB**主部从头开始编号，就是文件号。所有文件的**FCB**主部长度固定且相同，因此，给出文件号就可以计算出对应**FCB**的位置。
- 将**FCB**分为**FCB**主部和次部后，文件目录项中仅保存**FCB**的次部。这样，根据文件名查找文件目录可以找到文件控制块的次部，根据文件控制块次部得到的文件号就可以找到文件控制块的主部，进而找到文件。



文件目录的改进

- 将**FCB**分为主部和次部的优点：
 - 可以提高查找速度(顺序查找)：文件目录是存放在外存储器中的，需要以块为单位将其读入内存。由于一个**FCB**包括许多信息，这样一个外存块中所包含的**FCB**就会较少，导致查询速度较慢。将**FCB**分为主部和次部后，文件目录中仅保存**FCB**次部，一个外存块可以容纳较多的**FCB**次部，大大提高了文件的检索速度。
 - 可以实现文件连接(**link**)：所谓连接就是给文件起多个名字，这些名字都是路径名，为不同用户使用。

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

- 24.** 文件系统中，把**FCB**分为次部和主部的好处是
- A.** 提高文件的查找速度
 - B.** 减少**FCB**所占空间
 - C.** 防止进程修改**FCB**信息
 - D.** 减少文件**I/O**操作的时间

答案 A

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

25. 为保证磁盘文件安全，需要对磁盘文件进行转储。假设系统对磁盘文件进行了**3**次转储后，发生了磁盘数据丢失。下述对磁盘数据丢失进行恢复的论述正确的是

- A.** 完全转储策略只需要**2**盘转储磁带恢复磁盘数据；
- B.** 增量转储策略只需要**2**盘转储磁带恢复磁盘数据；
- C.** 差分转储策略只需要**2**盘转储磁带恢复磁盘数据；
- D.** 上述论述都不正确。

知识点： 文件系统的安全



文件系统的安全

■ Backup

- 定期将磁盘上文件复制到磁带上
- 发生故障时由磁带恢复(**limited recovery**)

■ 实现方法

- 完全转储(**full backup**)
 - 定期将磁盘上文件全部复制到磁带上
- 增量转储(**incremental backup**)
 - 每次只复制修改部分
- 差分转储(**differential backup**)
 - 初始时完全转储，之后改进增量转储。也就是说开始时，对系统进行一次完全转储，然后，再备份时将所有与开始第一次备份内容不同的数据备份到磁带上。



文件系统的安全

■ 完全转储(full backup)

- 优点：当发生数据丢失时，可以完全恢复。
- 缺点：定期备份，造成备份数据大量重复，占用大量磁盘空间，增加成本，另外，时间代价也比较大。

■ 增量转储(incremental backup)

- 优点：节省磁盘空间，缩短备份时间。
- 缺点：当发生数据丢失时，数据恢复比较困难。并且其可靠性差，各个备份磁带间的关系如同链子一样，一环套一环，其中任何一盘磁带出现问题都会导致整条链子脱节。

■ 差分转储(differential backup)

- 避免了完全转储和增量转储的缺点，并综合了二者的优点。因为差分转储不需要每次都对系统做完全转储，因而，备份所需时间短，并能节省磁带空间。灾难恢复方便，只需要2盘磁带，即第一次备份磁带和灾难发生前一次磁带，即可将系统完全恢复。

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

25. 为保证磁盘文件安全，需要对磁盘文件进行转储。假设系统对磁盘文件进行了**3**次转储后，发生了磁盘数据丢失。下述对磁盘数据丢失进行恢复的论述正确的是

- A.** 完全转储策略只需要**2**盘转储磁带恢复磁盘数据；
- B.** 增量转储策略只需要**2**盘转储磁带恢复磁盘数据；
- C.** 差分转储策略只需要**2**盘转储磁带恢复磁盘数据；
- D.** 上述论述都不正确。

答案 C

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

26. 假设操作系统利用缓冲技术在进程与打印机之间通过软缓冲区实现向打印机的输出，则该缓冲区的结构是

- A.** 外存连续空间队列
- C.** 内存连续空间队列

- B.** 外存链式队列
- D.** 内存链式队列

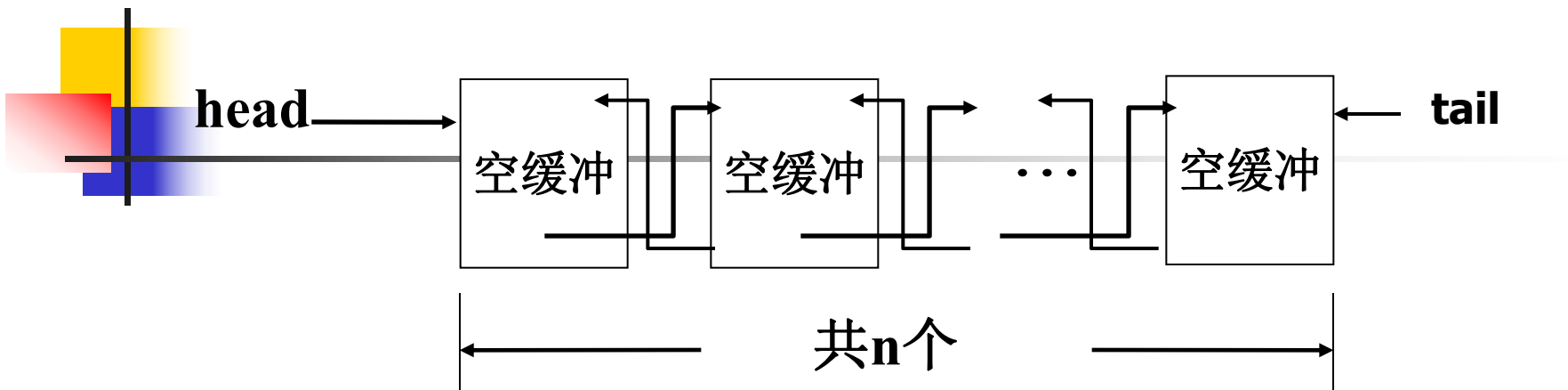
知识点：缓冲技术

缓冲技术

Buffering vs. Caching

- **buffering:** 缓冲，处理数据到达与离开速度不一致所采用的技术。**buffering**中的数据是没有副本的。
- **caching:** 高速缓存，为了减少访问磁盘次数而提出，以提高访问速度。**caching**中的数据在磁盘中是有副本的。
- 硬缓冲与软缓冲
 - 硬缓冲区通常设在设备中
 - 软缓冲区通常设在内存系统空间中（操作系统管理）
- 私用缓冲与公共缓冲
 - 私用缓冲：一个缓冲区与一个固定设备相联系，不同设备使用不同的缓冲区
 - 利用率低
 - 公用缓冲：缓冲区由系统统一管理，按需要动态分派给正在进行I/O传输的设备

缓冲池管理



Var buf_num:semaphore; (init n)

mutex:semaphore; (init 1)

1. 申请

(1) P(buf_num)

(2) P(mutex)

(3) 取链头空缓冲

(4) V(mutex)

2. 释放

(1) P(mutex)

(2) 空缓冲入链尾

(3) V(mutex)

(4) V(buf_num)

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

26. 假设操作系统利用缓冲技术在进程与打印机之间通过软缓冲区实现向打印机的输出，则该缓冲区的结构是

- A.** 外存连续空间队列
- C.** 内存连续空间队列

- B.** 外存链式队列
- D.** 内存链式队列

答案 D

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

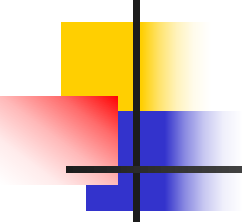
27. 在**RAID**数据存储标准中，既能进行并行读、又能有条件进行并行写的**RAID**级别是

A. Level 2	B. Level 3
C. Level 4	D. Level 5

知识点： **RAID**级别

RAID级别

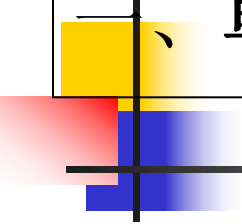
- **RAID**: 是一个物理磁盘的集合, 作为一个逻辑磁盘被管理和使用。数据被分散存于多个物理磁盘上
- **RAID级别**: 行业标准规定的数据在多个磁盘上的存放方法。
 - 常见RAID级别: level0, ..., level5;
- **RAID分条(striping)数据存储方式**
 - 位级分条(bit-level striping)
 - 块级分条(block-level striping)
- **RAID衡量指标**
 - **速度**: 是否支持多个访问同时进行;
 - **可靠性**: 是否能够发现和改正错误;
 - **成本**: 是否有额外的开销和开销的大小.



RAID级别(Cont.)

表8-1 RAID 级别的比较

Level	分条粒度	读并发性	写并发性	冗余(容错/开销)
0	块	支持	支持	无
1	块	支持	不支持	镜像
2	位	不支持	不支持	汉明纠错码奇偶校验与恢复
3	位	不支持	不支持	单个奇偶校验
4	块	支持	不支持	块级异或校验
5	块	支持	支持	块级分布式异或校验



一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

27. 在**RAID**数据存储标准中，既能进行并行读、又能有条件进行并行写的**RAID**级别是

A. Level 2	B. Level 3
C. Level 4	D. Level 5

答案 D

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

28. 假设某分布式操作系统采用分布式的 **Chandy-Misra-Haas** 算法进行死锁检测。当进程 **P_j** 接收到进程 **P_k** 发出的探测信令 **(i, k, j)** 这一时刻，系统出现了可能造成死锁的环路。则下面成立的式子是

A. $i=k$

B. $i=j$

C. $k=j$

D. $i \neq j$

答案 B （第**9**章分布式操作系统内容）

29. 分布式操作系统的文件系统中，对于文件访问的无状态服务有如下论述：

- I. **API**界面中不包含文件打开和关闭命令；
- II. 服务器端在内存文件控制表中应保持远程文件访问的控制信息；
- III. 每个文件读/写命令必须是自包含的(**self contained**)；
- IV. 打开文件数有限制。

其中正确的论述是

- A.** I 和III
- B.** I 和IV
- C.** II 和III
- D.** II 和 IV

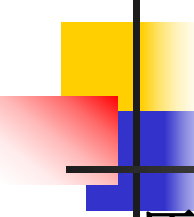
答案 A （第9章分布式操作系统内容）

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

30. 假设用户远程登录采用一次性口令。设单向函数为 $y=f(x)$ ，用户初次选定口令为 S ， S 使用次数为 n 。用户第 i 次登录时传送给主机 P_i ，主机验证用户登录口令要计算 $f(P_i)$ ，实际上与 $f(P_i)$ 相等的是

- A. $f^{n-i-1}(S)$ B. $f^{n-i-2}(S)$
C. $f^{n-i+1}(S)$ D. $f^{n-i+2}(S)$

知识点：一次性口令



一次性口令 (one time password)

原理

基于单向函数 $y=f(x)$

给定 x , 可以很容易地计算 y ;

给定 y , 从计算上来说不可能求得 x ;

用户首先选定一个保密口令 s , 同时指定一个整数 n (口令使用次数)

Password generation(用户产生的口令):

第一代口令为 $p_1=f^n(s)$;

第二代口令为 $p_2=f^{n-1}(s)$;

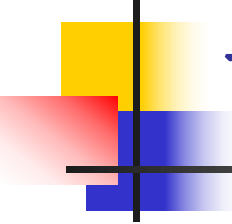
第三代口令为 $p_3=f^{n-2}(s)$;

...

第 n 代口令为 $p_n=f(s)$.

一次性口令

- 主机初始化
 - $P_0 = f(P_1) = f(f^n(s)) = f^{n+1}(s)$ 和 n 记在 password file 中
- 第一次登录：
 - 主机响应 n ,
 - 远程用户输入口令 s , 在客户端计算出 $p_1 = f^n(s)$ 并传送给主机,
 - 主机计算出 $p_0 = f(p_1)$ 并将其与 password file 中的 p_0 相比较.
 - 如果相同登录成功, 主机用 p_1 取代 password file 中的 p_0 , 并将 n 减 1.



一次性口令

- 第二次登录：
 - 主机响应 $n-1$,
 - 远程用户输入口令 s , 在客户端计算出 $p_2 = f^{n-1}(s)$ 并传送给主机 ,
 - 主机计算出 $p_1 = f(p_2)$ 并将其与 **password** 文件中的 p_1 相比较 .
 - 如果相同登录成功, 主机用 p_2 取代 **password** 文件中的 p_1 , 并将 n 减 **1** .
- 特点
 - 抗截取



用户每次输入的口令
不变, 都是 s

一、单项选择题（共**30**小题，每小题**1**分，共**30**分）

30. 假设用户远程登录采用一次性口令。设单向函数为 $y=f(x)$ ，用户初次选定口令为 S ， S 使用次数为 n 。用户第 i 次登录时传送给主机 P_i ，主机验证用户登录口令要计算 $f(P_i)$ ，实际上与 $f(P_i)$ 相等的是

A. $f^{n-i-1}(S)$

B. $f^{n-i-2}(S)$

C. $f^{n-i+1}(S)$

D. $f^{n-i+2}(S)$

答案 D

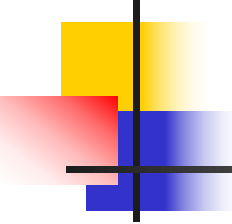
二、（进程调度，10分）

已知四个进程 P1、P2、P3、P4 到达就绪队列时间和 CPU 阵发时间如下（时间单位：毫秒）：

Process	Arrival time	Burst time
P1	0	7
P2	2	4
P3	4	9
P4	13	5

问题：(1) 画出按可抢占短作业优先 CPU 调度算法得到的进程调度结果的甘特图；

(2) 计算四个进程的平均等待时间、平均周转时间、平均带权周转时间。



短作业优先：按照**CPU**阵发时间递增的次序调度

周转时间：完成时间-进入时间

$$T = t_f - t_s$$

平均周转时间：周转时间的平均值

$$\bar{T} = \frac{1}{n}(\sum_{i=1}^n T_i)$$

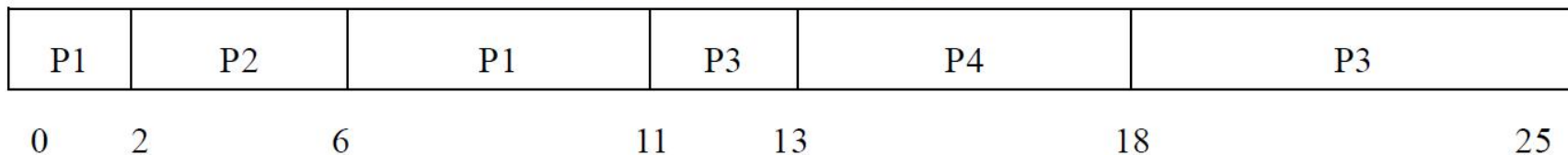
带权周转时间：周转时间/运行时间

$$W = \frac{T}{R}$$

平均带权周转时间：带权周转时间的平均值

$$\bar{W} = \frac{1}{n}(\sum_{i=1}^n W_i) = \frac{1}{n}(\sum_{i=1}^n \frac{T_i}{R_i})$$

解：(1) (4 分) 按可抢占 SJF 调度算法的进程调度结果甘特图：



(2) (6 分) 四个进程的平均等待时间、平均周转时间、平均带权周转时间见下表：

进程	到达时间	运行时间	开始时间	完成时间	等待时间	周转时间	带权 周转时间
P1	0	7	0	11	4	11	11/7
P2	2	4	2	6	0	4	1
P3	4	9	11	25	12	21	21/9
P4	13	5	13	18	0	5	1

平均等待时间 = $(4+0+12+0) / 4 = 16/4 = 4\text{ms}$

平均周转时间 = $(11+4+21+5) / 4 = 41/4 = 10.25\text{ms}$

平均带权周转时间 = $(11/7+1+21/9+1) / 4 = 372/63 \times 4 \approx 1.475\text{ms}$

三、（死锁静态分析，10分）

设有 7 个简单资源：A、B、C、D、E、F、G，各资源申请命令分别为 a、b、c、d、e、f、g，释放命令分别为 $\bar{a}$ 、 $\bar{b}$ 、 $\bar{c}$ 、 $\bar{d}$ 、 $\bar{e}$ 、 $\bar{f}$ 、 $\bar{g}$ 。设系统有 P1、P2、P3 三个进程，各进程有关资源的活动序列为：

P1 活动：a b $\bar{a}$ $\bar{b}$ e f g $\bar{e}$ $\bar{f}$ $\bar{g}$

P2 活动：b c $\bar{b}$ $\bar{c}$ d a $\bar{d}$ $\bar{a}$

P3 活动：c d $\bar{c}$ $\bar{d}$ g f e $\bar{e}$ $\bar{f}$ $\bar{g}$

分析当 P1、P2、P3 并发执行时，是否有发生死锁的可能性，并说明原因。

知识点：可复用资源死锁的静态分析

条件：已知各个进程有关资源的活动序列；

判断：有无死锁可能性。

步骤1：以每个进程占有资源，申请资源作为一个状态，
记作：(pi: aj: ak1,...,akn)=(进程：请求：占有)；

步骤2：以每个状态为一个节点；

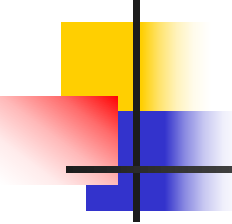
步骤3：如s1所申请资源为s2所占有，则由s1向s2画一有向弧（相同进程间不画）；

步骤4：找出所有环路；

步骤5：判断环路上状态是否能同时到达，如有死锁可能性，否则无死锁可能性。

(1)环路中有相同进程，不能到达；

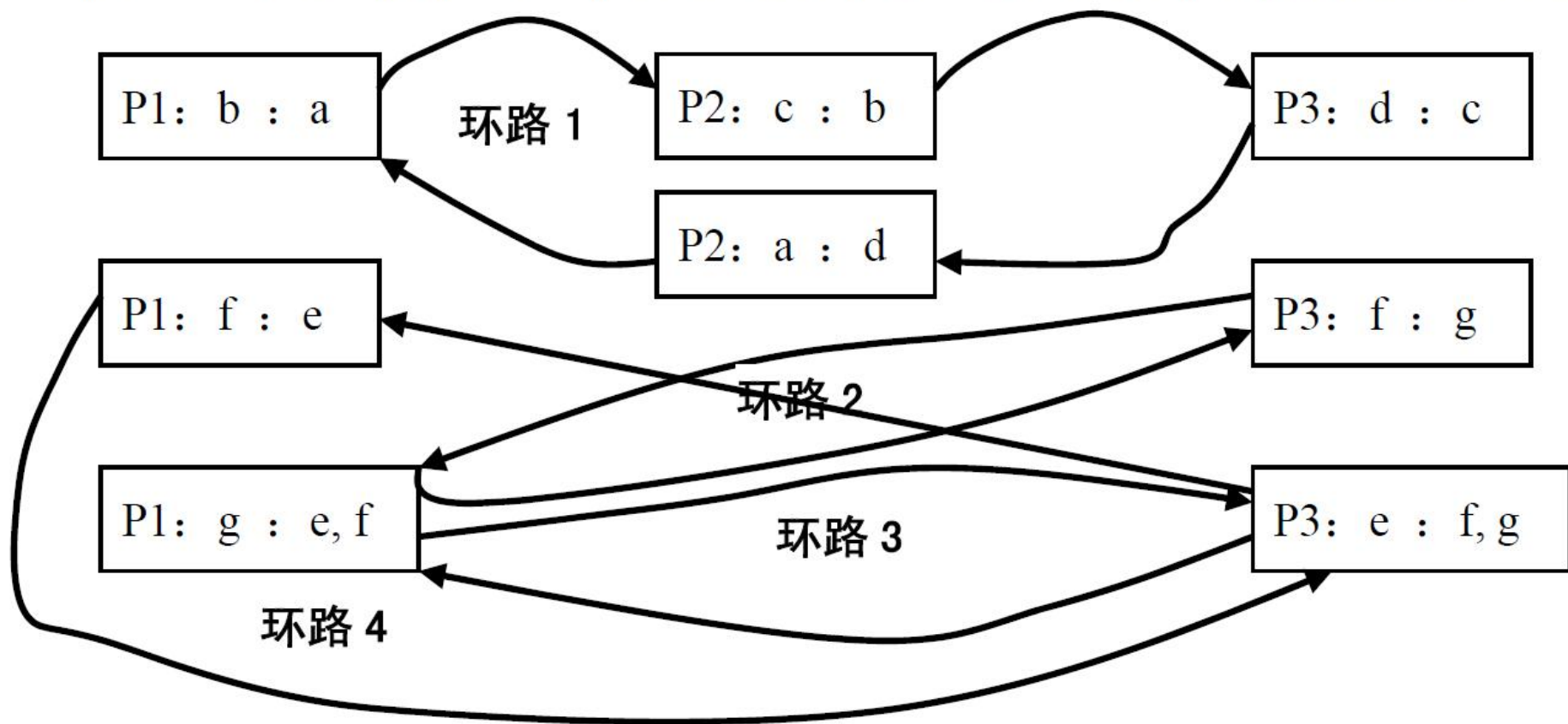
(2)环路中有相同被占有资源，不能到达。



知识点：死锁

- 死锁定义：一组进程中的每一个进程，均无限期地等待此组进程中某个其他进程占有的，因而永远无法得到的资源，这种现象称为进程死锁。
- 死锁发生的条件：
 - 资源独占
 - 不可掠夺
 - 保持申请
 - 循环等待

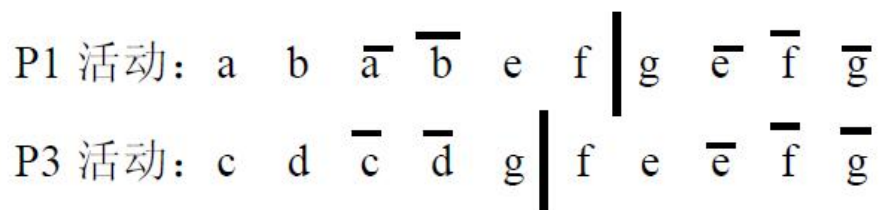
解：(1) (4 分) 根据三个进程有关资源的活动，画出进程资源申请、占用状态图如下：



(2) (6分) 环路分析:

环路 1: 该环路中有相同的进程 P2, 三个进程并发执行时, 不会发生此种情况的死锁;

环路 2: 该环路反映的 P1、P3 的运行情况是两个进程同时执行到竖线位置 (如下图)

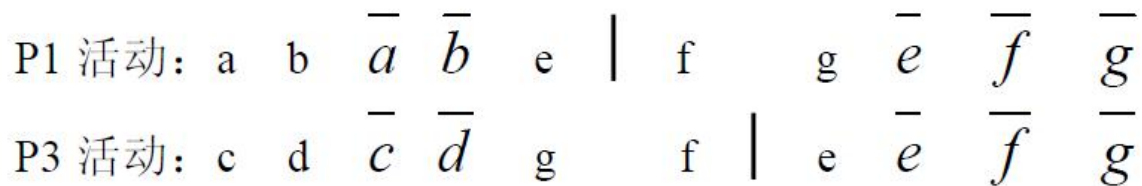


两个进程能同时执行到竖线位置, 此时

P1 占有资源 E 和 F, 要申请资源 G; P3 占有资源 G, 要申请资源 F。出现死锁;

环路 3: 该环路反映的运行状态是进程 P1 和 P3 同时占有资源 F, 但由于资源 F 只有一个, 因此此种运行状态不可能出现, 即不会发生此种情况的死锁。

环路 4: 该环路反映的 P1、P3 的运行情况是两个进程同时执行到竖线位置 (如下图)



两个进程能同时执行到竖线位置, 此时

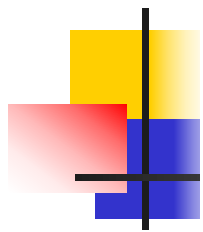
P1 占有资源 E, 要申请资源 F; P3 占有资源 G、F, 要申请资源 E。出现死锁

四、（内存管理，10分）

设某计算机主存有**16**个页框，内存分配和释放采用伙伴堆算法(**Buddy heap algorithm**)。主存空闲区表的表项**free_area[i]**的结构为：

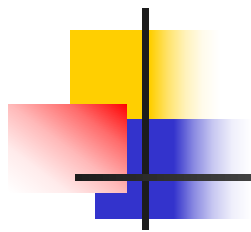
空闲块组i的位图指针	空闲块组i的指针
------------	----------

空闲块组链的结点结构包括前、后结点指针和本空闲块组的首页框号**3**个数据域。设当前主存映像图及主存空闲区表**free_area**格式如下：



主存映像	页框号
占用	0
	1
占用	2
	3
	4
	5
占用	6
占用	7
	8
	9
	10
	11
	12
占用	13
	14
	15

free_area	
块组 号 i	块组位 图指针
0	
1	
2	
3	
4	



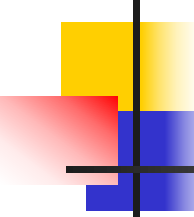
问题：

- (1) 根据主存映像图，写出伙伴堆算法的主存空闲区表**free_area**各表项的空闲块组链表和块组位图指向的块组位图内容的数据结构；
- (2) 基于 (1) 所做的数据结构图，设有长度为**3**页的内存申请，写出按伙伴堆算法分配页架后的主存映像图和伙伴堆算法数据结构图。



知识点:伙伴堆(Linux 存储管理)

- **Linux** 采用**DMA**方式进行输入输出操作，**DMA**不带有地址变换机构，即是在没有地址映射的条件下进行的，因此进程在**内存中**必须占有连续的页面。
- 从地址映射角度，页式存储管理方法并不要求一个进程所分得的多个页面在物理上连续，不适用于**Linux**
- 针对**linux**系统，对**内存**空闲页面管理时，需要将连续的页面放在一组，这就是伙伴堆的思想。
- 伙伴堆算法用于管理内存中的空闲块，它是针对**内存**碎片问题而提出的一种稳定高效的分配策略。



1. 伙伴堆存储分配算法

(1) Physical memory management

- 页框: 静态等长, 4KB;
- 块组: **Linux**将所有空闲页面分为**10**个块组, 块组编号为 $i(i=0,1\dots9)$, 块组 i 中记载长度为 2^i 个页面的连续区域, 即第**0**组中块的大小为 2^0 (1页), 第**1**组中块的大小均为 2^1 (2页), 第**9**组中块的大小均为 2^9 为(512页), 同组中的所有块以链表形式存储。
- 空闲区表: **free_area[i]**表示页框数为 2^i 的块组, 其结构为:

空闲块组指针	块组位图指针
--------	--------
- 分配/释放: **Buddy heap algorithm**
以 2^i 个页框(块组)为分配/释放单位($2^{i-1} < fn \leq 2^i$),
fn为要申请的页框数;



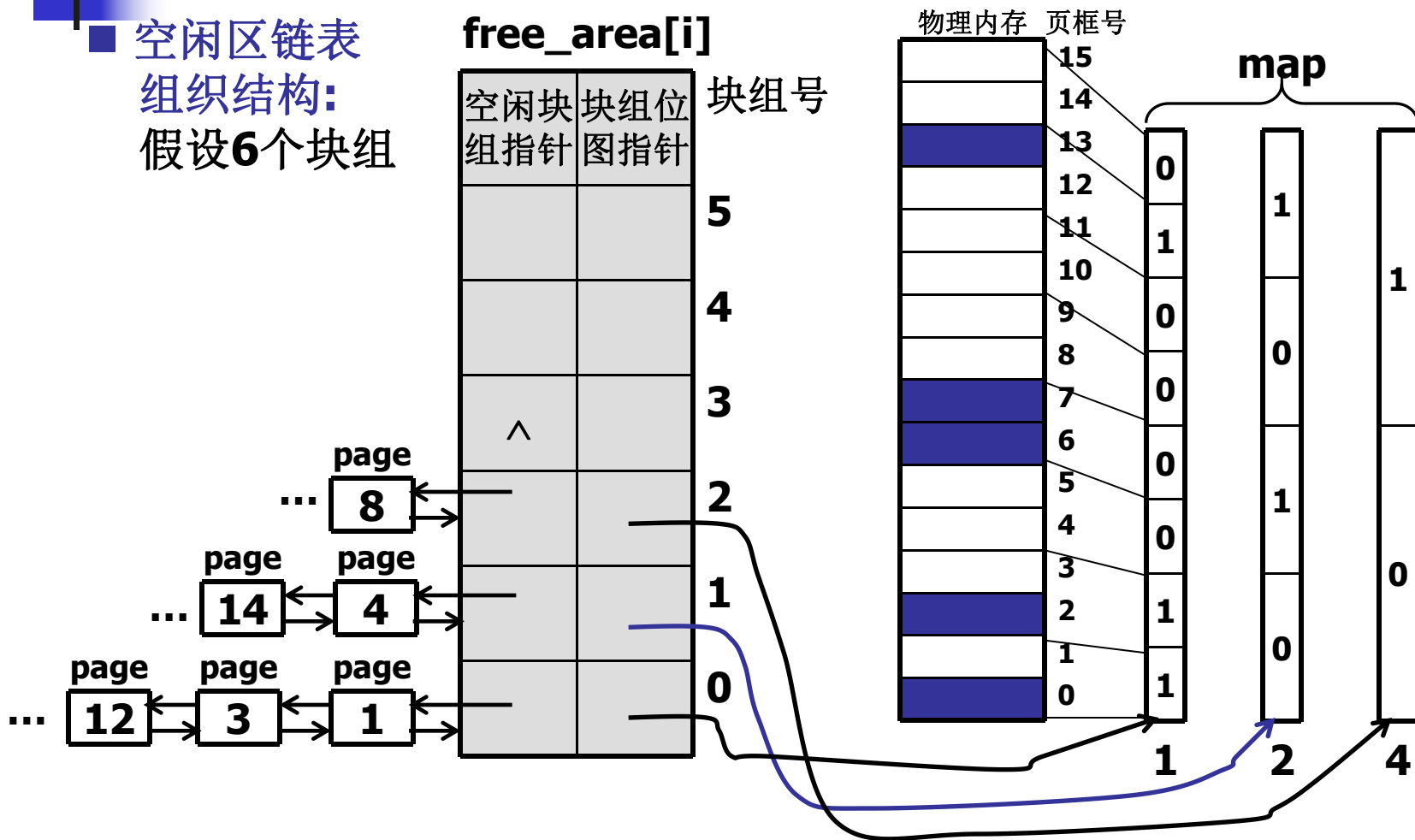
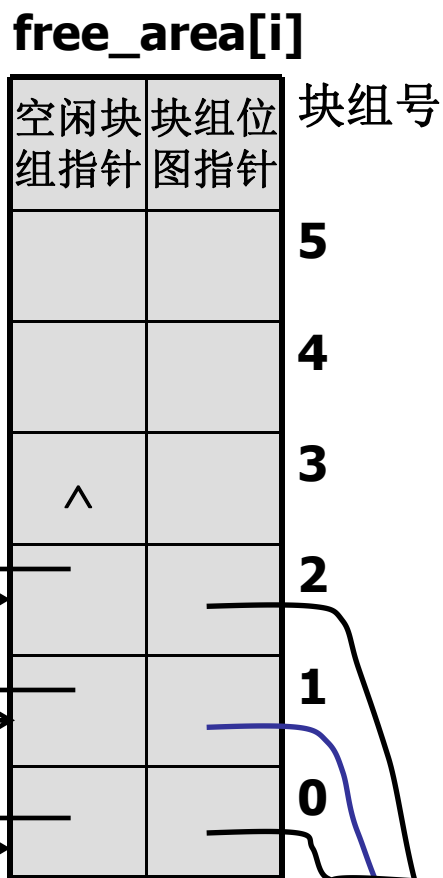
■ 块组位图:

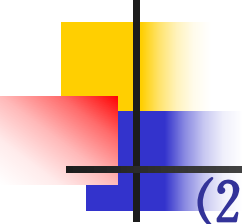
- 对于块组 i , 将内存中的所有页面(包括占用和空闲), 按前后顺序两两结合成一对伙伴(**Buddy**). 即按前后顺序以 2^i 个页面作为一块, 与其相邻的 2^i 个页面作为一块, 那么这两块为一对**Buddy**. 如: 2^1 块组的0、1页框和2、3页框是一对**Buddy**;
- 块组位图的 1 位表示对应的一对**Buddy**页框块组的使用情况;
- 对于一对**Buddy**:
 - ① 若一个空闲, 另一个全部或部分占用, 则位图相应位置1;
 - ② 当两个都空闲, 或都被全部或部分占用, 则位图相应位置0。

■ 伙伴条件:

- ① 两个块大小相同, 即具有相同的页框数 b ;
- ② 两个块的物理地址相连;
- ③ 如果以0作为页框的初始编号, 那么位于后面块组的最后页框编号+1必须是 $2b$ 的整数倍。

■ 空闲区链表 组织结构： 假设6个块组





(2) Buddy heap algorithm

- **分配:** 申请 fn 个页框

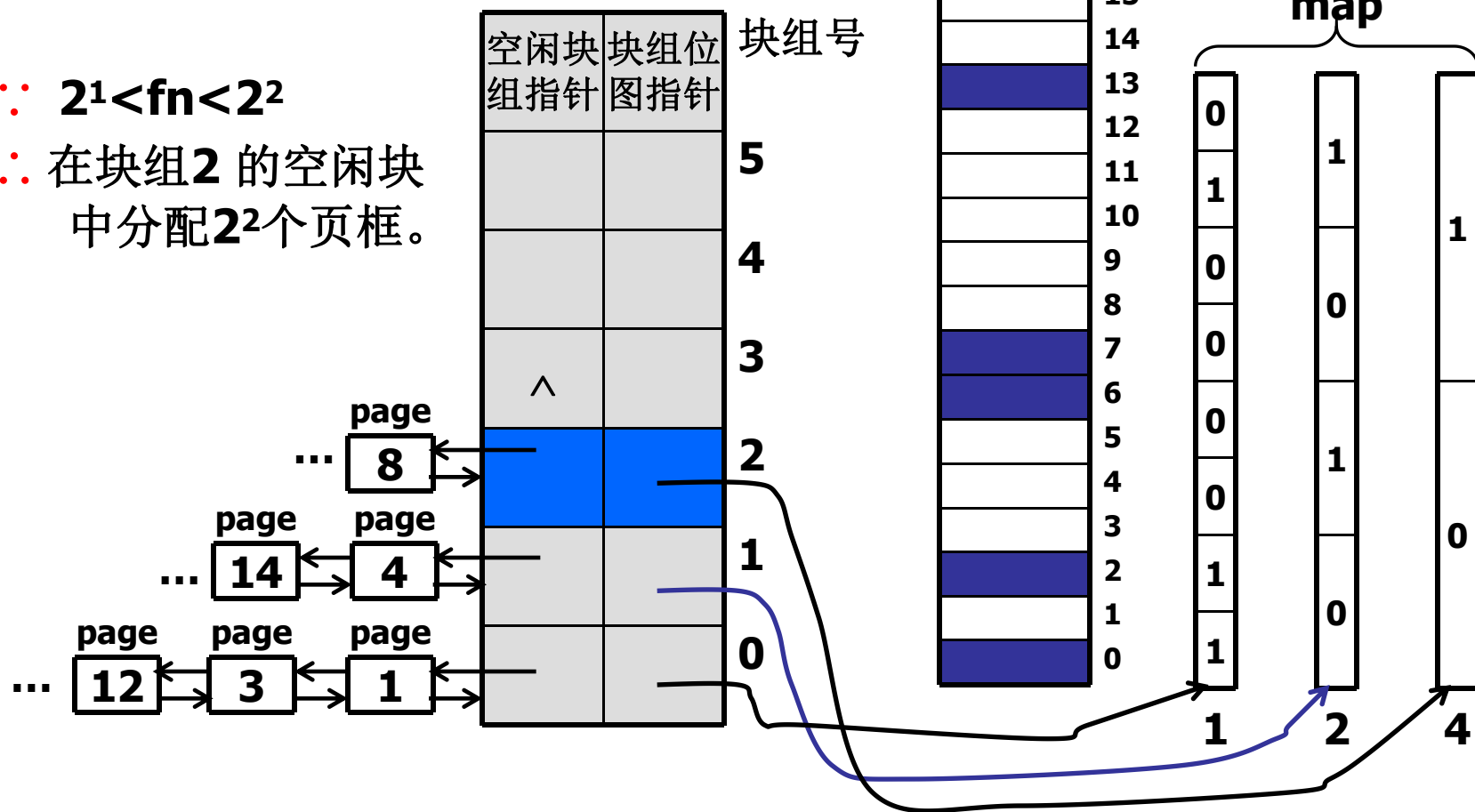
- ① 找到相应的块组 j ;
- ② 在块组 j 的第一个空闲块分配 2^i 个页框($2^{i-1} < fn \leq 2^i$);
- ③ 调整块组 j 的空闲块链表;
- ④ 若 $2^j > 2^i$, 则把 $2^j - 2^i$ 个空闲页框加入到相应块组空闲链中;
(若 $2^j - 2^i$ 不是2的整数次幂, 则将其拆分成不同的整数次幂。)
- ⑤ 修改位图。

例如: 对于长度为**128**页的请求, 应该在第**7**组中取一块分配。如果第**7**组已空, 取第**8**组中的一块, 分配其中的**128**页, 并将剩余的**128**页加入第**7**组中。若第**8**组也空, 取第**9**组中的一块, 进行两次分割, 分配**128**页, 将剩余的**128**页和**256**页分别计入第**7**组和第**8**组。

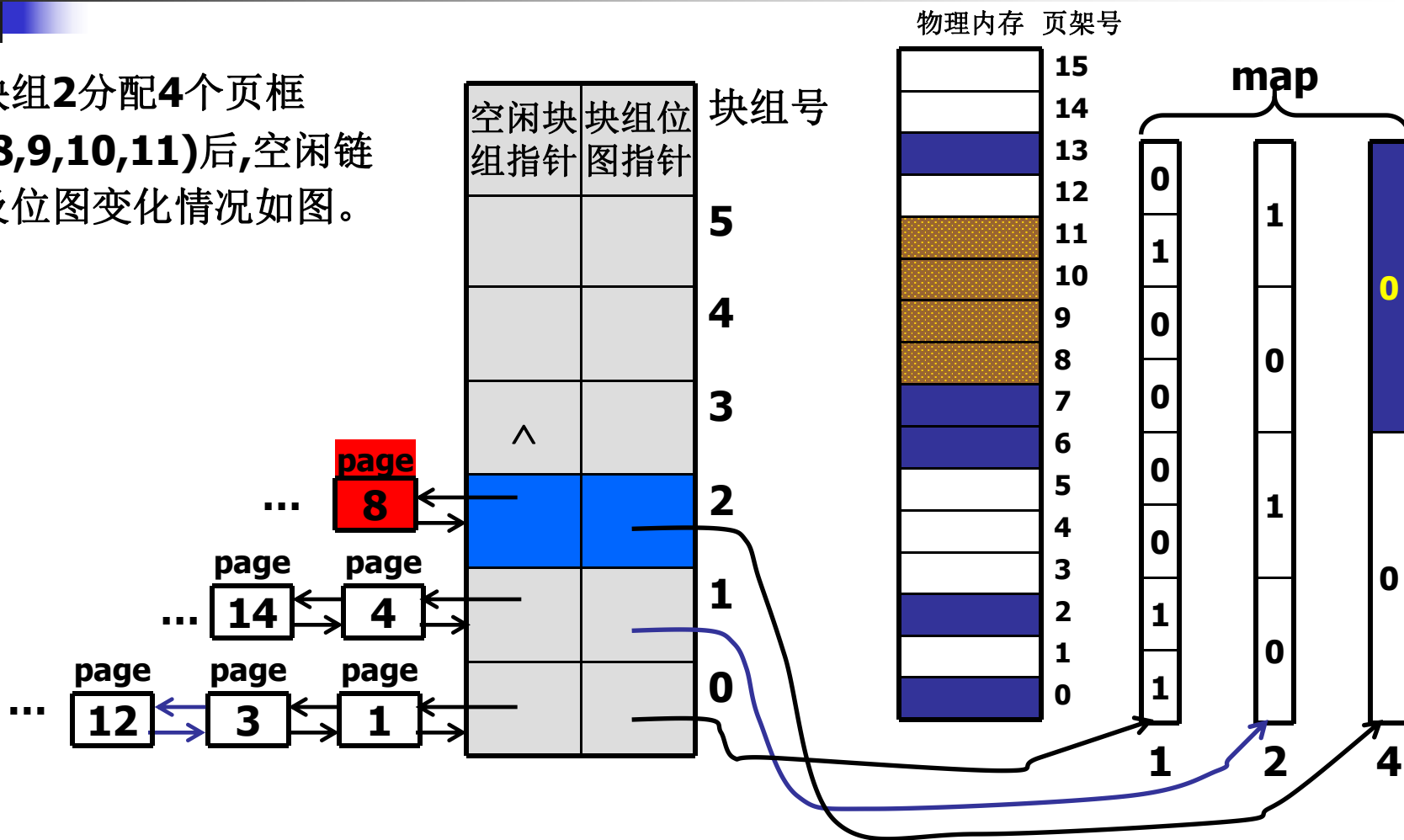
分配例: 申请页框数 $fn=3$

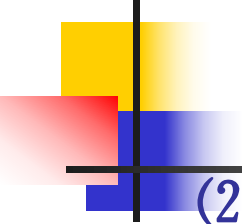
∴ $2^1 < fn < 2^2$

∴ 在块组2的空闲块中分配 2^2 个页框。



块组2分配4个页框
(8,9,10,11)后,空闲链
及位图变化情况如图。





(2) Buddy heap algorithm

- **释放:** 释放 2^i 个页框
 - ① 释放的 2^i 个页框与相邻的空闲区按伙伴关系合并, 即两个相邻的伙伴合并为一个大的空闲区;
 - ② 把得到的空闲区加入到不同块组的空闲链中;
 - ③ 修改位图。

■ 释放例：释放页框13

释放13后：

12,13是伙伴，

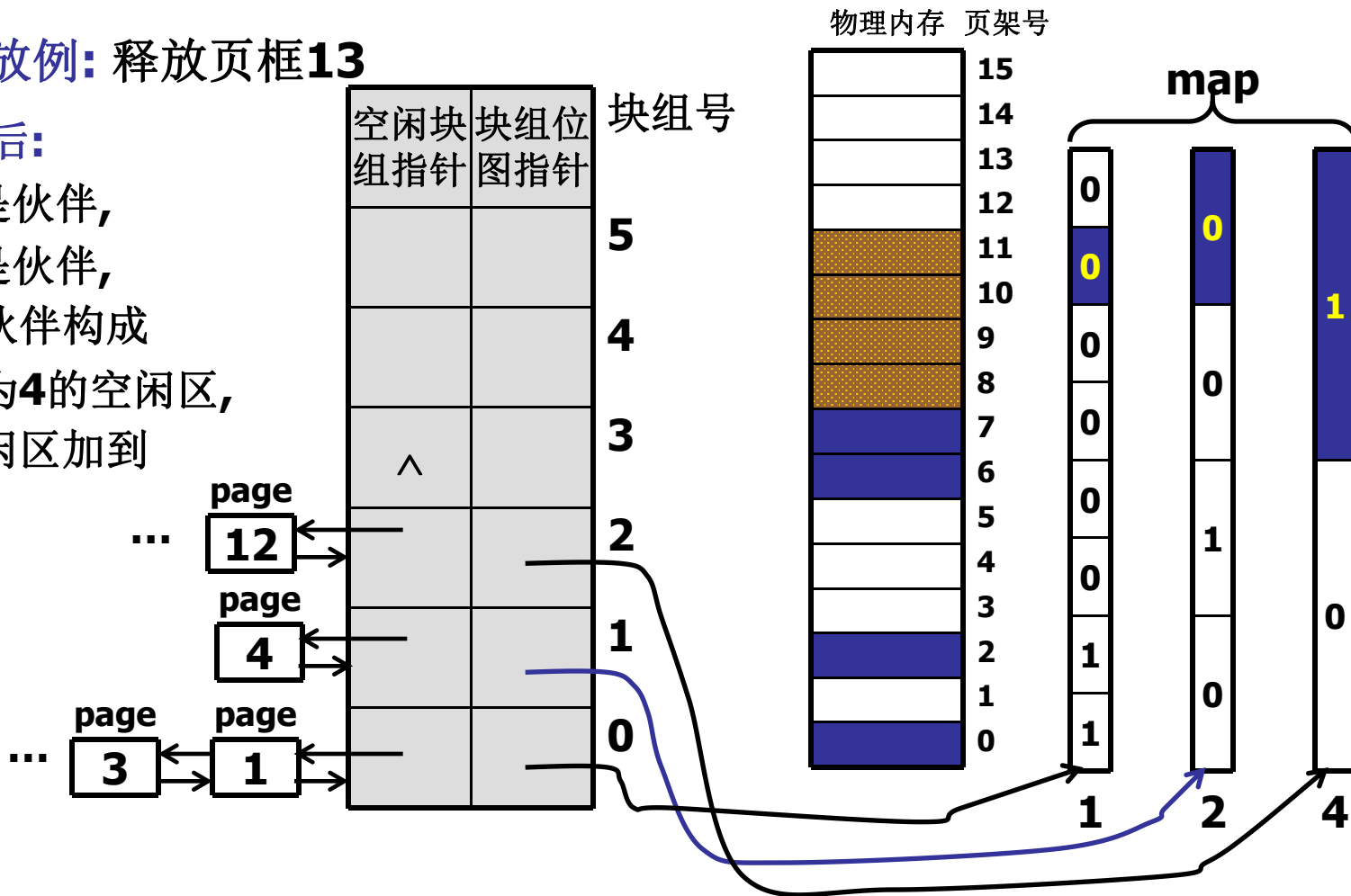
14,15是伙伴，

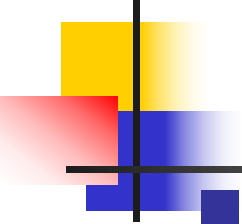
这两个伙伴构成

页框数为4的空闲区，

将该空闲区加到

块组2。





■ Buddy heap algorithm:

- **问题:** internal fragmentation.

例如: 申请17个页架, 由于 $2^4 < 17 \leq 2^5$,
按Buddy heap algorithm,
要在块组5的空闲区分配32个页框,
造成15个页框的浪费, 即internal fragmentation.

- **解决办法:**

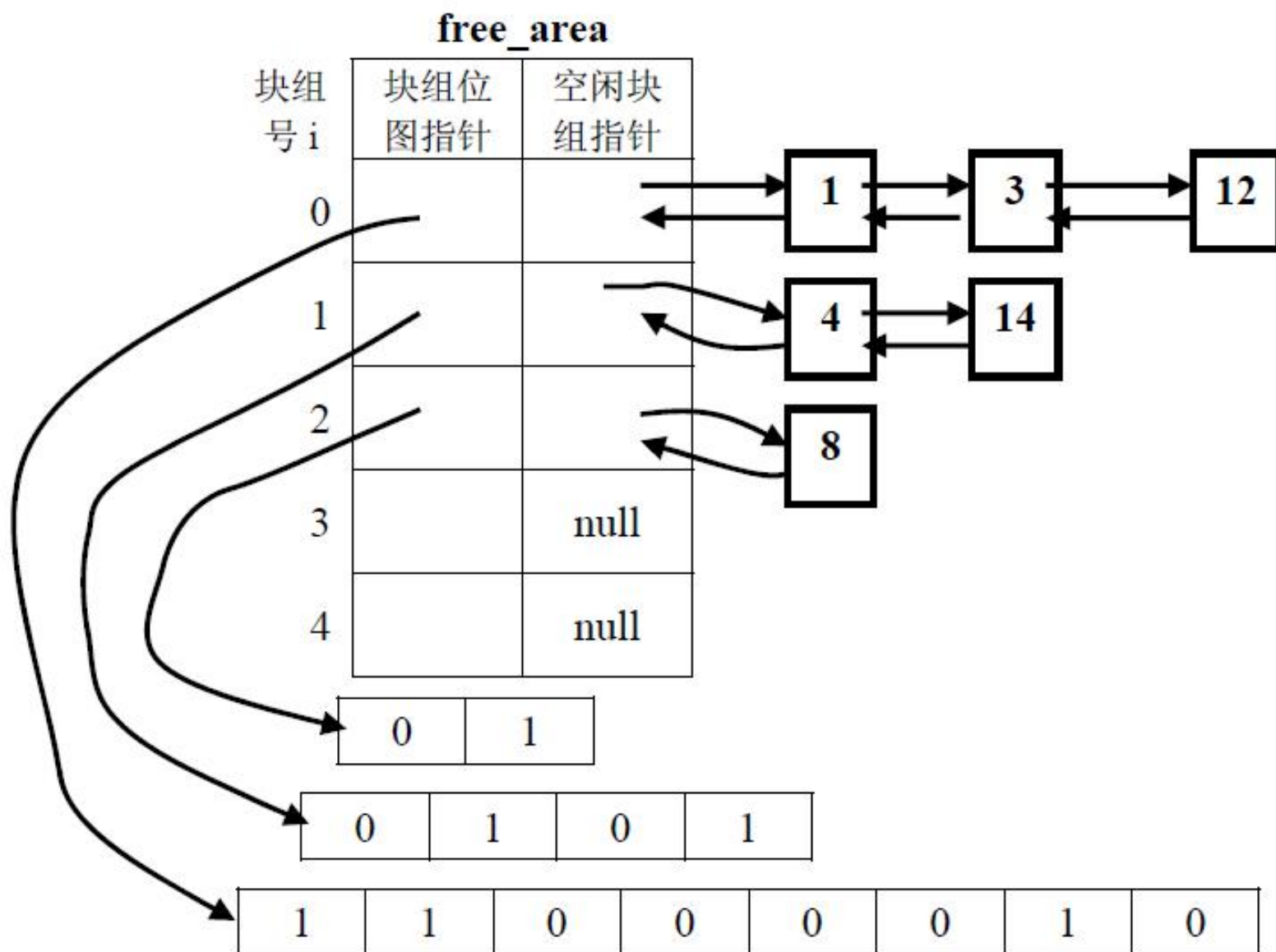
- ① **second memory allocator**

当实际申请页框数 $fn < 2^i$ 时,
将 $2^i - fn$ 按2的整数次幂切分(carves slabs),
由second memory allocator单独管理。

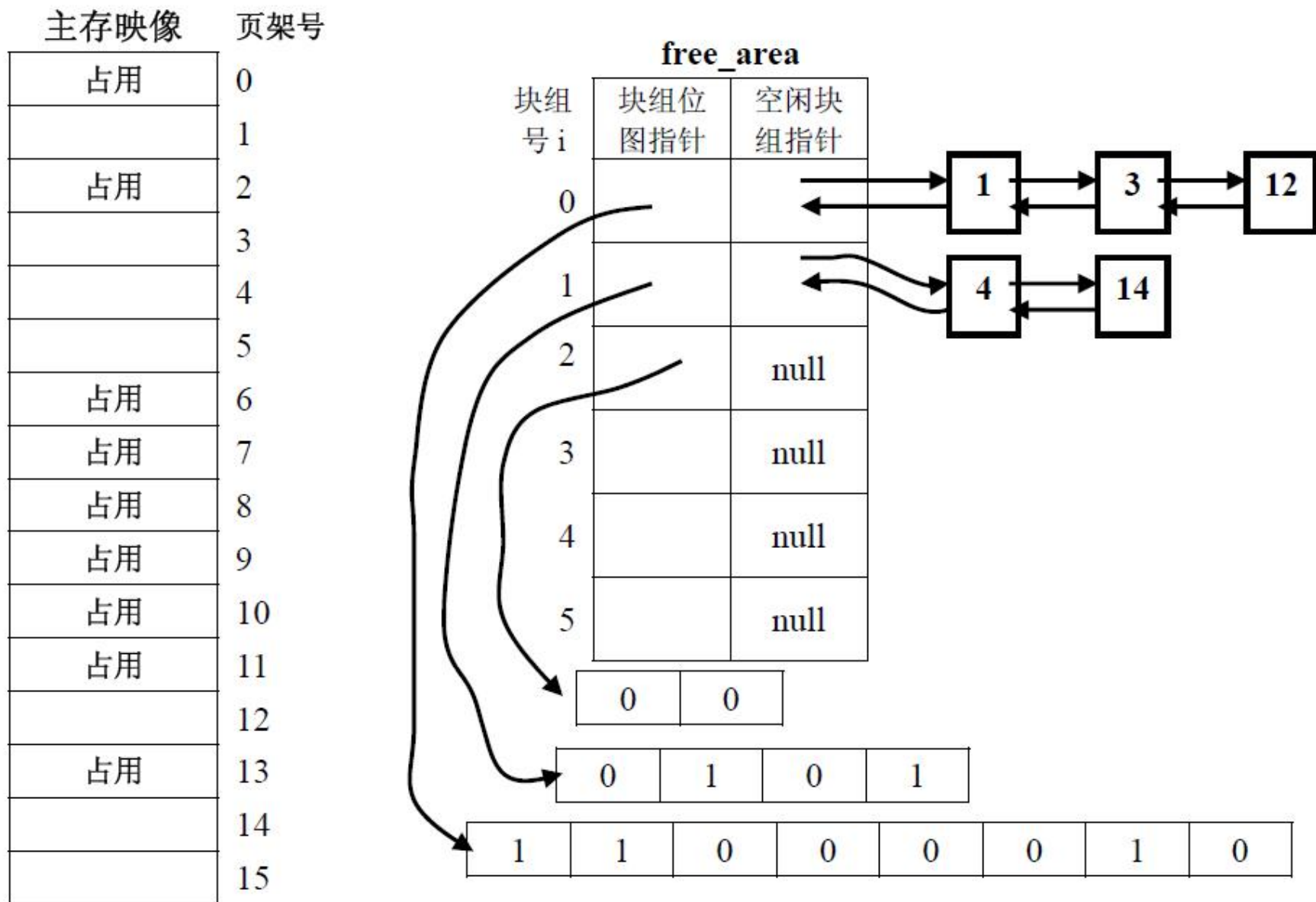
- ② **third memory allocator**

进程物理空间不要求连续时,
内存分配由third memory allocator完成。

解：(1) (5 分) 伙伴堆算法的空闲块组链表和块组位图的数据结构图：



(2) (5 分) 按伙伴堆算法响应申请后的主存映像图和数据结构:

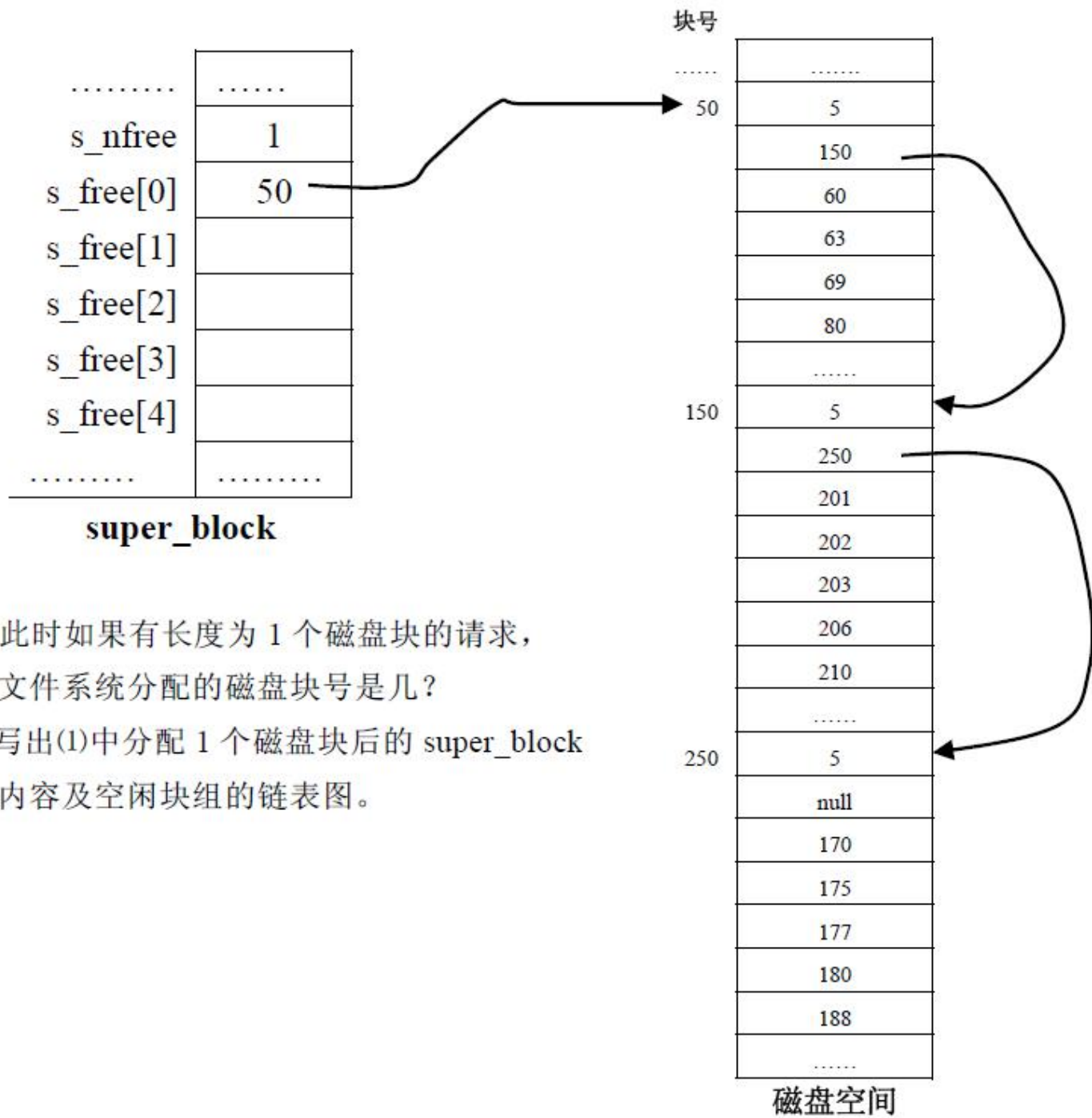


五、（磁盘管理，10分）

某文件系统的磁盘空闲区管理采用成组链接数据结构，涉及的数据结构类 C 伪代码定义如下：

```
struct filesys {  
    .....  
    int  s_nfree;    /* 本空闲块组中空闲块个数，最大为 5。*/  
    int  s_free[5];  /* s_free[0]存放下一空闲块组的块号；s_free[1]~s_free[4]指向本组的块号。*/  
    .....  
    } super_block;  /*系统启动后将磁盘空闲块组链的第一个空闲块组表读入 super_block.*/
```

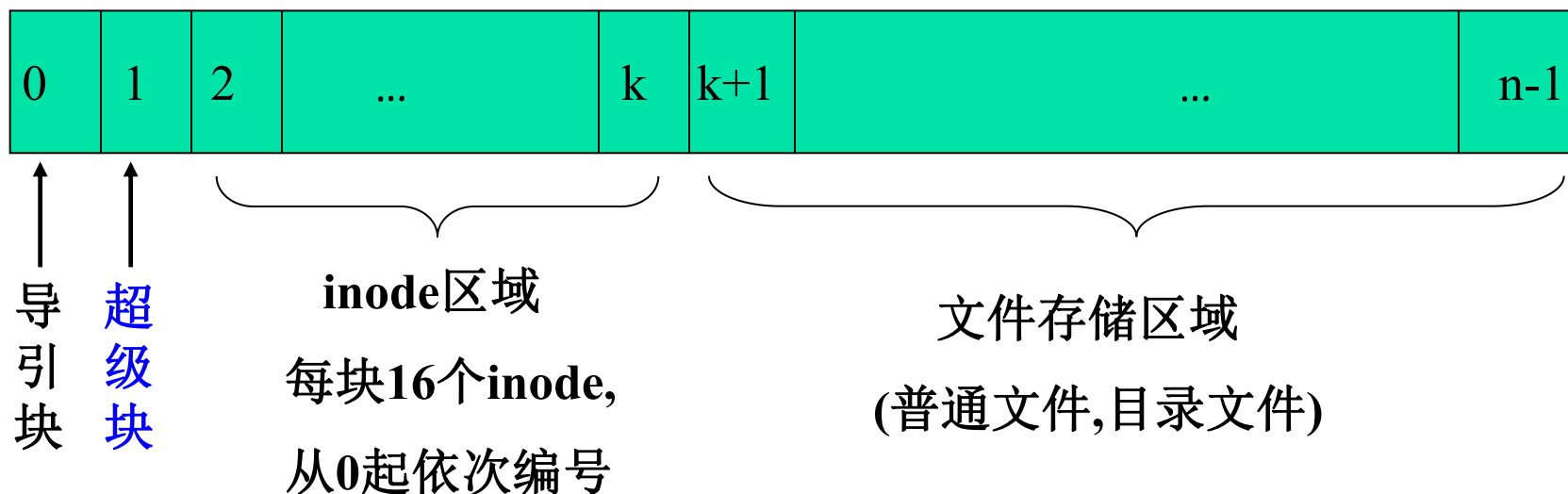
成组链接数据结构描述的当前磁盘空闲区如下图所示：

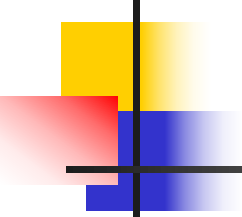


- 问题：(1) 此时如果有长度为 1 个磁盘块的请求，文件系统分配的磁盘块号是几？
- (2) 写出(1)中分配 1 个磁盘块后的 **super_block** 内容及空闲块组的链表图。

知识点:引导块和超级块

UNIX文件卷

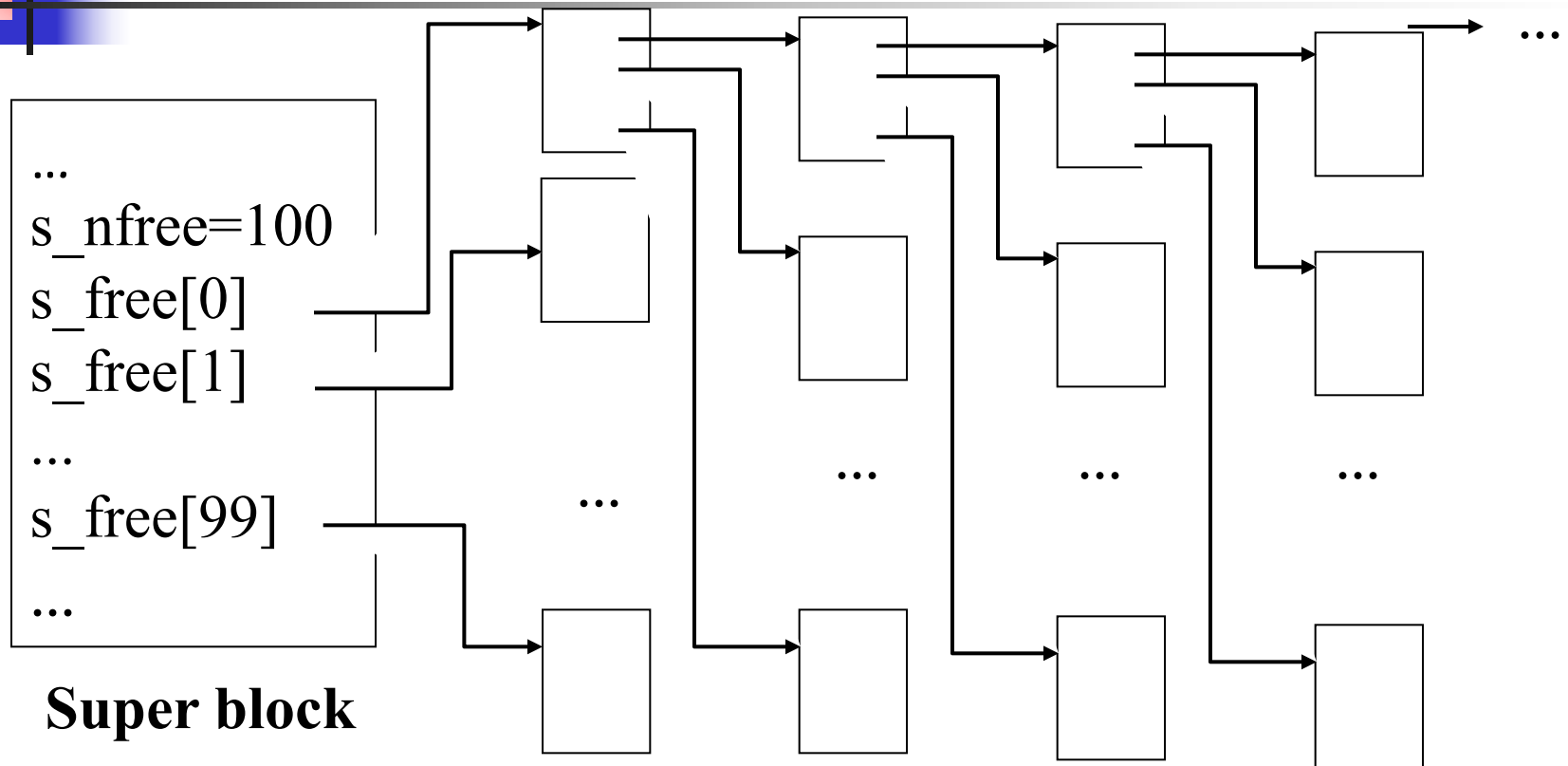


- 
- 块0#(引导块): 负责在系统启动时从磁盘上找到UNIX系统并将其装入内存
 - 块1#(super block): 是一个文件卷中最重要的数据结构,
 - (1) 记载文件卷上k+1块到n-1块中所有空闲块,
 - (2) inode区中100个空闲inode. (缓冲)
 - 文件安装(mount)后超级块读入内存。
 - 注: 占用区域已经记载在各个文件的inode中。

超级块结构（安装后读入内存，卸下时回写磁盘）

```
Struct fileysys {  
    int s_isize;    //size in blocks of i list  
    int s_fsize;    //size in blocks of entire volume  
    int s_nfree;    //number of in core free blocks  
    int s_free[100]; //in core free blocks  
    int s_ninode;    //number of in core I list  
    int s_inode[100]; //in core free I nodes  
    char s_flock;    //free list locking  
    char s_iloc;    //i list locking  
    char s_fmod;    //super block modified flag  
    char s_ronly;    //mounted read only flag  
    char s_time[2];    //current date of last update  
    int pad[50];  
}
```

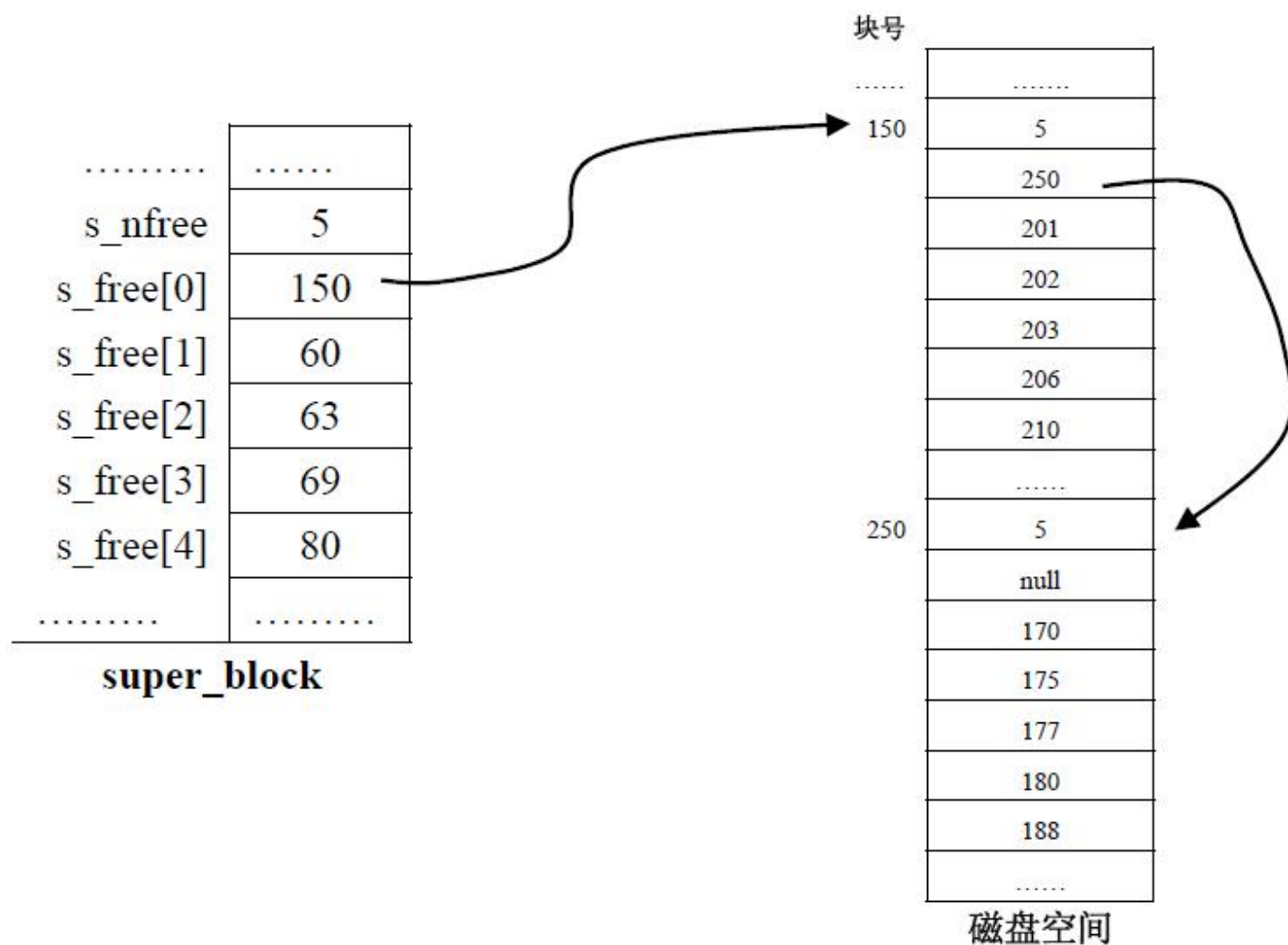
100个空闲块为一组，组之间相互链接。



特点：速度快，空间省。 P394页图13-14

解：(1) (4 分) 分配的磁盘块号是 50;

(2) (6 分) super_block 内容及空闲块组链表图如下:



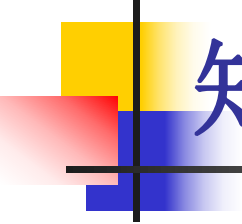
六、（信号量与P/V操作，10分）

设系统有多个生产者进程向**1**个缓冲区不断发消息，**n**个消费者进程从该缓冲区取消息。该缓冲区大小只能存放**1**条消息，并且对每条放入缓冲区的消息，所有消费者必须都接收**1**次，生产者才可以继续向缓冲区发消息。

问题：用信号量和**P**、**V**操作完成生产者进程和消费者进程发、收消息的正确描述。

要求：

- (1) 给出信号量变量的定义、初值及其含义；
- (2) 实现对缓冲区及消息操作的互斥与同步；
- (3) 用类**C**语言伪代码描述。



知识点:信号灯个**PV**操作

信号灯个**PV**操作题解思路

- 1** 先找出活动体;
- 2** 写出活动体的工作过程;
- 3** 找出相关活动体之间的合作关系;
- 4** 需要等待的地方在之前执行**P**操作, 需要唤醒的地方在之后执行**V**操作;
- 5** 有几个资源类那么定义几类信号量。



用信号灯实现进程同步

General Case:

VAR S:semaphore; (initial value 0)

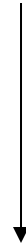
P1:



P(S)
后动作

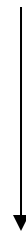


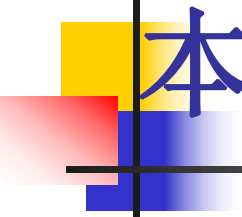
P2:



先动作

V(S)





本题分析

1 先找出活动体-----生产者、消费者

2 写出活动体的工作过程。

生产者

```
{ while(1)
```

```
{
```

生产产品

放入缓冲区

```
}
```

```
}
```

消费者

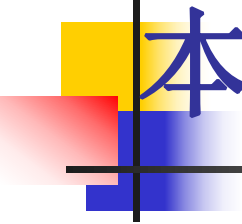
```
{ while(1)
```

```
{
```

从缓冲区取产品

消费

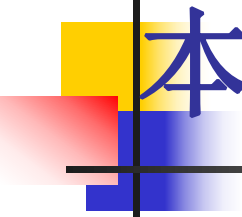
```
}
```



本题分析

3 找出相关活动体之间的合作关系(条件)。

- ①生产者在向缓冲区中放入产品时需要判断是不是所有消费者都取过产品。
- ②消费者在取产品之前，要判断缓冲区中是否有产品。
- ③生产者如果生产产品后放入缓冲区后需要唤醒消费者。
- ④消费者取完产品后需要通知生产者已经取完。



本题分析

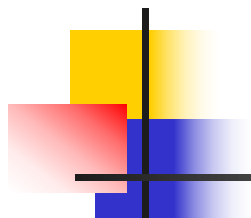
4 需要等待的地方在之前执行**P**操作，需要唤醒的地方在之后执行**V**操作。

生产者

```
{ while(1)
{
    生产产品
    P(S1) (条件①)
    放入缓冲区
    V(S2) (条件③)
}
}
```

消费者

```
{ while(1)
{
    P(S2)(条件②)
    从缓冲区取产品
    V(S1)(条件④)
    消费
}
}
```



5 有几个资源类定义几个信号量。

semaphore mutex;

//初值为**1**，用于缓冲区读、写互斥；

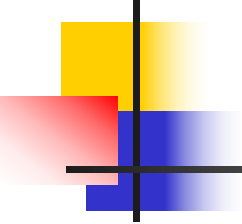
semaphore S1; //初值为**1**；用于生产者判断是否有空闲缓冲区；

semaphore S2[n]; //即**S2**

//初值均为**0**；用于生产者写入消息后,唤醒消费者实现与**n**个消费者同步；

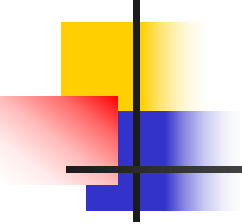
message buffer;

//假设**message**为消息类型，**buffer**为缓冲区；



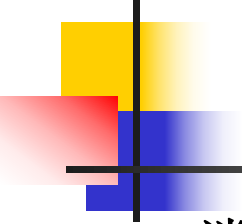
解法1: (1) (4分) 信号量变量定义:

```
semaphore mutex=1; S1=1;  
S2[n]=0(n=1,2.....n);  
message    buffer;  
int count=0;
```



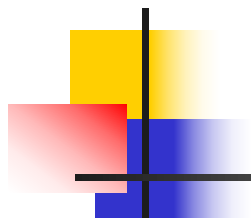
生产者进程:

```
void producer( )  
{int i; message ms;  
  while(1){  
    ms=produce();  
    P(S1);  
    P(mutex);  
    write(buffer,ms);  
    V(mutex);  
    for(i=0,i<n,i++) V(S2[i])  
  }  
}
```

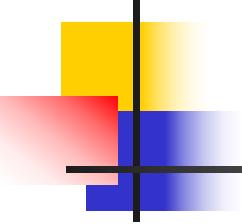
消费者进程:

```
void consumer(process_id i)  
  {int i; message ms;  
    while(1){P(S2[i]);  
        P(mutex);  
        ms=read(buffer);  
        count=count+1;  
        if count==n then V(S1)  
        V(mutex);  
        consume(ms);  
    }  
}
```



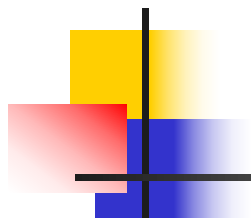
5 有几个资源类定义几个信号量。

```
semaphore mutex;  
//初值为1，用于缓冲区读、写互斥；  
semaphore read_n[n]; // 即S1  
//初值均为1；用于生产者判断n个消费者均读完消息；  
semaphore write_n[n]; //即S2  
//初值均为0；用于生产者写入消息后,与m个消费者同步；  
message buffer;  
//假设message为消息类型，buffer为缓冲区；
```



解发2: (1) (4分) 信号量变量定义:

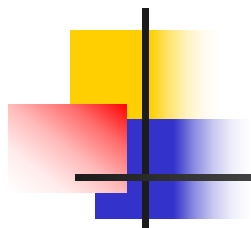
- **semaphore mutex;** //初值为**1**, 用于缓冲区读、写互斥;
- **semaphore read_n[n];** //初值均为**1**; 用于生产者判断**n**个消费者均读完消息;
- **semaphore write_n[n];** //初值均为**0**; 用于生产者写入消息后, 与**m**个消费者同步;
- **message buffer;** //假设**message**为消息类型, **buffer**为缓冲区;



(2) (6分) 生产者、消费者进程描述:

生产者进程:

```
void producer( )
{ int i; message ms;
  while(1){
    ms=produce(); // “生产” 一条消息;
    for(i=0;i<n;i++) P(&read_n[i]); //所有消费者都读完上次发的消息?
                                     //如有消费者还没读消息, 则生产者等待。
    P(&mutex); write(buffer,ms); V(&mutex); //把消息 ms 写入缓冲区 buffer;
    for(i=0;i<m;i++) V(&write_n[i]); //通知所有消费者, 已经把消息写入缓冲区;
  }
}
```



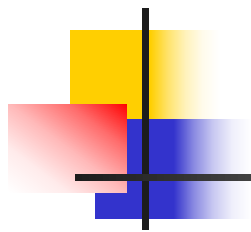
消费者进程:

```
void consumer(process_id k)    //k 为消费者进程号,  $0 \leq k \leq n-1$ ;  
{ int i; message ms;  
  while(1){  
    P(&write_n[k]); //生产者已发消息? 如没发, 则等待;  
    P(&mutex); ms=read(buffer); V(&mutex); //把缓冲区 buffer 中消息读到 ms;  
    V(&read_m[k]); //通知生产者: 消费者进程 k 已经读完缓冲区中的消息;  
    consume(ms) //消费者“消费”消息 ms;  
  }  
}
```

七、（虚拟存储管理，10分）

某虚拟页式存储管理，页长为**512 Bytes**，工作集固定为**4**，缺页中断时基于局部置换策略并采用二次机会（**Second Chance**）淘汰算法。内存空间为**64 KB**，进程最大空间为 **64 KB**。设进程**P**当前页表（**page table**）如下：

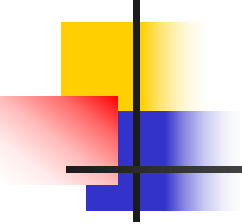
Page table			
逻辑页号	页框号	访问标志	装入时间(ms)
4	3	0	90
6	8	1	80
9	5	1	60
12	13	0	70



问题:

(1) 进程**P**访问虚拟地址**0C9EH**和**15DBH**时, 访问的逻辑页号分别是多少?

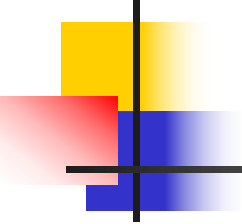
(2) 问题(1)中两个虚拟地址访问对应的物理页框号和物理地址分别是多少?



解：(1) (4分)

0C9EH转换二进制为0000,1100,1001,1110B，由于页长为**512bytes=29bytes**，故页内地址为低**9**位，高**7**位为逻辑页号，则**0C9EH**访问的逻辑页号为**6**；

同理，**15DBH**=0001,0101,1101,1011B，**15DBH**访问的逻辑页号为**10**。



(2) (6分) 根据页表,

- 虚拟地址**0C9EH**:

不发生缺页中断, 访问的物理页框号=**8**, 访问的物理地址=**0001,000 0,1001,1110B**=**109EH**

- 虚拟地址**15DBH**:

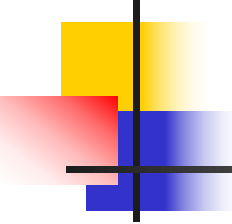
发生缺页中断, 根据二次机会淘汰算法, 应淘汰页面**12**, 其对应的页框号为**13**, 即把页面**10**装入到**13**号页框。因此访问的物理页架号=**13**, 访问的物理地址=**0001,101 1,1101,1011B**=**1BDBH**



八、（设备与I/O管理，10分）

设系统磁盘只有一个移动磁头，磁道由外向内编号为：**0、1、2、.....、199**；磁头移动一个磁道所需时间为**1毫秒**；每个磁道有 **100** 个扇区，每个扇区**128Bytes**；磁盘转速 **$R=6000r/min$** . 当前磁头位置处于第**102**磁道，当前移动方向由外向内。设有磁道的**I/O**请求序列：**70、130、50、112、125、120、80、30、60、40、90、20、110**，每个请求读/写磁道上的**1**个扇区；系统对磁盘设备的**I/O**请求采用**5步扫描(5-step Scan)**调度算法。问题：

- (1) 写出给定**I/O**请求序列的调度序列，并计算磁头的移动量
- (2) 对给定**I/O**请求序列，计算总寻道时间（启动时间忽略）、总旋转延迟时间、总传输时间和总访问处理时间。



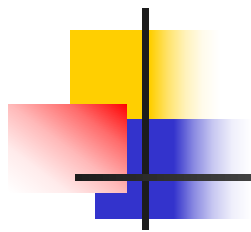
知识点: **N步扫描**和**SCAN**扫描

■ **N-step SCAN** (**N步扫描**)

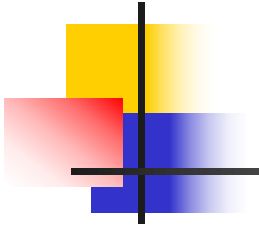
- 将磁盘请求队列分为若干个长度为**N**的子队列, 每个队列内采用**SCAN**算法
- 当**N**很大时, 接近**SCAN**算法
- 当**N=1**时, 蜕化为**FCFS**算法

■ **SCAN** (扫描算法)

- 扫描算法往复扫描各个柱面(磁道)并为路径柱面的请求服务。起始时, 磁头处于最外柱面, 并向内柱面移动。在移动的过程中, 如果路径的柱面有访问请求, 则为其服务, 如此一直移动到最内柱面, 然后改变方向由最内柱面向外柱面移动, 并以相同的方式为路径的请求服务。扫描算法每次扫描到柱面的尽头, 无论最内(外)柱面是否有访问请求。



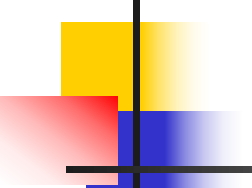
解：(1) (4分) 调度序列为(不包括括号内的磁道):
(102)→112→125→130→(199)→70→50→(0)
→30→40→60→80→120→(199)→110→90→20
磁头移动量=**(112-102)+(125-112)+(130-**
125)+(199-130)+(199-70)+(70-50)+(50-
0)+(30-0)+(40-30)+(60-40)+(80-60)+(120-
80)+(199-120)+(199-110)+(110-90)+(90-20)
=10+13+5+69+129+20+50+30+10+20+2
0+40+79+89+20+70=674 (磁道)



(2) ① 寻道时间 (seek time) : 将磁盘引臂移动到指定柱面所需要的时间。

由于总寻道数**674**，磁头移动一个磁道所需时间为**1**毫秒，那么：

$$\begin{aligned}\text{总寻道时间} &= \text{总寻道数} * \text{磁头移动一个磁道时间} \\ &= \mathbf{674 * 1 = 674ms}\end{aligned}$$



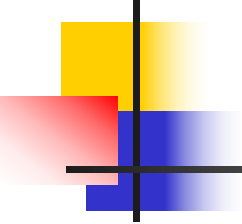
②旋转延迟 (**rotational delay**)：指定扇区旋转到磁头下的时间。旋转延迟 T_r 计算公式如下：

$$T_r = 1/(2r)$$

其中， r 为磁盘转速。该公式给出的是平均旋转延迟，它是磁盘旋转一周时间的一半，即旋转半周所花费的时间

$$\begin{aligned} \text{一次旋转延迟时间} &= 1/(2R) = 1/(2 \times 6000/\text{min}) \\ &= 1/(2 \times (100/\text{s})) = 0.005(\text{秒}) \\ &= 5\text{ms} \end{aligned}$$

由于请求序列共**13**次访盘，
总旋转延迟时间 $= 5 \times 13 = 65(\text{ms})$



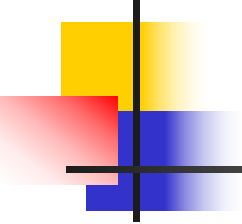
③传输时间 (**transfer time**)：读/写一个扇区的时间。传输时间**Tt**计算公式如下：

$$\mathbf{Tt=b/(rN)}$$

其中，**b**为读/写字节数，**r**为磁盘转速，**N**为一条磁道上的字节数。那么

$$\begin{aligned}\mathbf{1次访盘的传输时间}&=\mathbf{128/(R\times 100\times 128)} \\&=\mathbf{1/((6000/m)\times 100)} \\&=\mathbf{1/((100/s)\times 100)=0.1ms}\end{aligned}$$

$$\mathbf{13次访盘总传输时间=0.1\times 13=1.3毫秒}$$

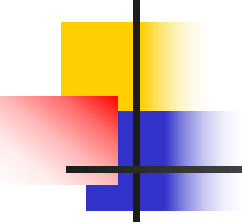


④总访问处理时间=总访盘处理时间+总旋转延迟时间+总传输时间
=674+65+1.3=740.3毫秒

在银行家算法中，若出现下面的资源分配情况：

Process	Allocation	Need	Available
P0	0032	0012	1622
P1	1000	1650	
P2	1354	2356	
P3	0032	0652	
P4	0014	0656	

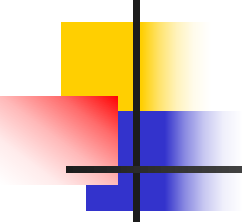
1622为资源ABCD的
数目



(1)、该状态是否安全？

安全状态是指系统的一种状态，在此状态下，系统能按某种顺序（例如 P_1 、 $P_2 \cdots P_n$ ）来为各个进程分配其所需资源，直至最大需求，使每个进程都可顺序地一个个地完成。这个序列（ P_1 、 $P_2 \cdots P_n$ ）称为**安全序列**。

若系统此状态不存在一个安全序列，则称系统处于**不安全状态**。



(1) 安全性算法

1. Let *Work* and *Finish* be vectors of length m and n , respectively. Initialize (让*Work*和*Finish*作为长度为 m 和 n 的向量)

***Work* := Available**

***Finish* [i] = false for $i = 1, 3, \dots, n$.**

2. Find an i such that both: (找到 i)

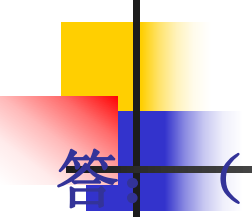
(a) ***Finish* [i] = false**

(b) **$Need_i \leq Work$**

If no such i exists, go to step 4.

3. ***Work* := *Work* + *Allocation* _{i}**
***Finish* [i] := true**
go to step 2.

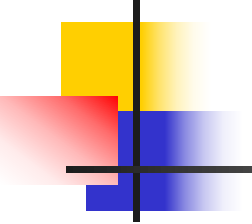
4. If ***Finish* [i] = true** for all i , then the system is in a safe state, otherwise in an unsafe state.



(1)、该状态是否安全？

答：(1) 利用安全性算法对上述状态进行分析，找到了一个安全序列 (**P0, P3, P4, P1, P2**)，故该状态是安全的。分析如下表所示。

process	work	Need	Allocation	Work+allocation	finish
P0	1622	0012	0032	1654	TRUE
P3	1654	0652	0032	1686	TRUE
P4	1686	0656	0014	169 10	TRUE
P1	169 10	1650	1000	269 10	TRUE
P2	269 10	2356	1354	39,14 , 14	TRUE



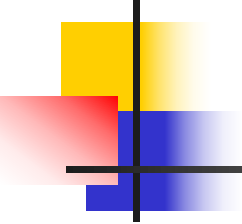
(2) 资源请求算法原理

当用户申请一组资源的时候，系统必须确定这些资源的分配是否仍会使系统处于安全状态。

如果会，就可以分配资源；否则，系统必须等待直到某个其他进程释放足够的资源为止。

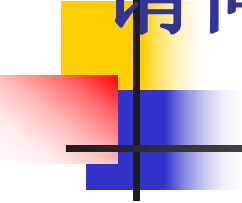
(2) 资源请求算法

1. If $Request_i \leq Need_i$, go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim.
2. If $Request_i \leq Available$, go to step 3. Otherwise P_i must wait, since resources are not available.
3. 假定给 P_i 分配资源
 $Available := Available - Request_i$
 $Allocation_i := Allocation_i + Request_i$
 $Need_i := Need_i - Request_i$
4. 用安全算法进行检查,看系统是否处于安全状态
 - If safe $\Rightarrow$ the resources are allocated to P_i .
 - If unsafe $\Rightarrow P_i$ must wait, and the old resource-allocation state is restored



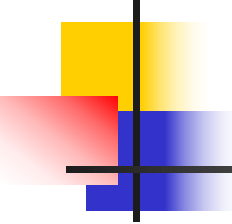
(2) 若进程P2提出请求Request (1, 2, 2, 2) 后, 系统能否将 资源分配给它?

- 解: P2 发出请求Request (1, 2, 2, 2) 后, 系统按银行家算法进行检查:
- Request (1, 2, 2, 2) $\leq$ Need2(2,3,5,6), 且 Request (1, 2, 2, 2) $\leq$ Available(1,6,2,2)
- 系统假定可为P2分配资源,并修改Available、Allocation2和Need2如下:
- Available= (0, 4, 0, 0)
- Allocation2= (2, 5, 7, 6)
- Need2= (1, 1, 3, 4)
- 接下来进行安全性检查: 此时对所有的进程, 条件 $Need_i \leq Available(0,4,0,0)$ 都不成立,故系统进入不安全状态.因此,不能给P2分配资源.



**(3) 如果系统立即满足P2的上述请求，
请问，系统是否立即进入死锁状态？**

解：系统立即满足进程**P2**的请求**(1,2,2,2)**后,并没有马上进入死锁状态.因为此时上述进程并没有申请新的资源，只是进入了不安全状态。只有当上述进程提出新的资源请求并导致所有没执行完的多个进程因得不到资源而阻塞时，系统才进入死锁状态。

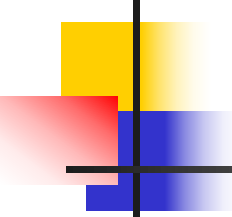


页面置换算法

- 在一个请求分页存储管理的系统中，一个程序的页面走向为6,0,1,2,0,3,0,4,2,3,分别采用最佳置换算法、先进先出置换算法、最近最久未使用算法计算缺页中断次数。设分配给该程序的存储块数 $M=3$, 每调进一个新页就发生一次缺页中断。

OPT页面置换算法

时刻	1	2	3	4	5	6	7	8	9	10
访问顺序	6	0	1	2	0	3	0	4	2	3
M=3	6	6	6	2	2	2	2	2	2	2
		0	0	0	0	0	0	4	4	4
			1	1	1	3	3	3	3	3
f	1	2	3	4		5		6		

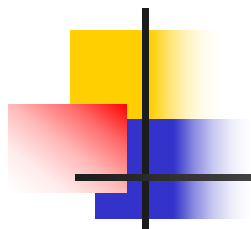


FIFO页面置换算法

时刻↵	1↵	2↵	3↵	4↵	5↵	6↵	7↵	8↵	9↵	10↵
访问顺序↵	6↵	0↵	1↵	2↵	0↵	3↵	0↵	4↵	2↵	3↵
M=3↵	6↵	6↵	6↵	2↵	2↵	2↵	2↵	4↵	4↵	4↵
	↵	0↵	0↵	0↵	0↵	3↵	3↵	3↵	2↵	2↵
	↵	↵	1↵	1↵	1↵	1↵	0↵	0↵	0↵	3↵
f↵	1↵	2↵	3↵	4↵	↵	5↵	6↵	7↵	8↵	9↵

LRU页面置换算法

时刻	1	2	3	4	5	6	7	8	9	10
访问顺序	6	0	1	2	0	3	0	4	2	3
M=3			1	2	0	3	0	4	2	3
		0	0	1	2	0	3	0	4	2
	6	6	6	0	1	2	2	3	0	4
f	1	2	3	4		5		6	7	8



结束